

INNOPOLIS UNIVERSITY

Введение в компьютерное зрение

Илья Афанасьев, к.т.н., ЛИРС

Лекция 9 · Дескрипторы и сопоставление признаков

Dr. Ilya Afanasyev, i.afanasyev@innopolis.ru

«Introduction to Computer Vision» course, Innopolis University (IU), Russia, Fall 2026

Карта курса

Блок I Изображение и его источник	**Блок II** Операции над изображением	**Блок III** Признаки	**Блок IV** Гомография и геометрия	**Блок V** Видео и движение
L1 Введение. Изображение и видео как данные	**L4** Геометрические преобразования	**L7** Границы, линии и робастное оценивание	**L10** Гомография: оценивание и выравнивание	**L13** Видео: движение и оптический поток
L2 Формирование изображения: оптика и сенсор	**L5** Пространственная фильтрация	**L8** Точки интереса: детекторы	**L11** Модель камеры и калибровка	**L14** Трекинг и распознавание объектов
L3 Цвет, гистограммы и точечные операции	**L6** Частотная область и пирамиды	**L9** Дескрипторы и сопоставление признаков	**L12** Стерео и эпиполярная геометрия	**L15** Синтез: от двух видов к 3D. Итоги

Сегодня — L9: Дескрипторы и сопоставление

1 Детектор фитч SIFT в OpenCV

SIFT (Scale-Invariant Feature Transform) · особые точки · разность гауссиан (DoG) · пирамида масштабов · 128-мерный дескриптор

The properties of a good feature detector

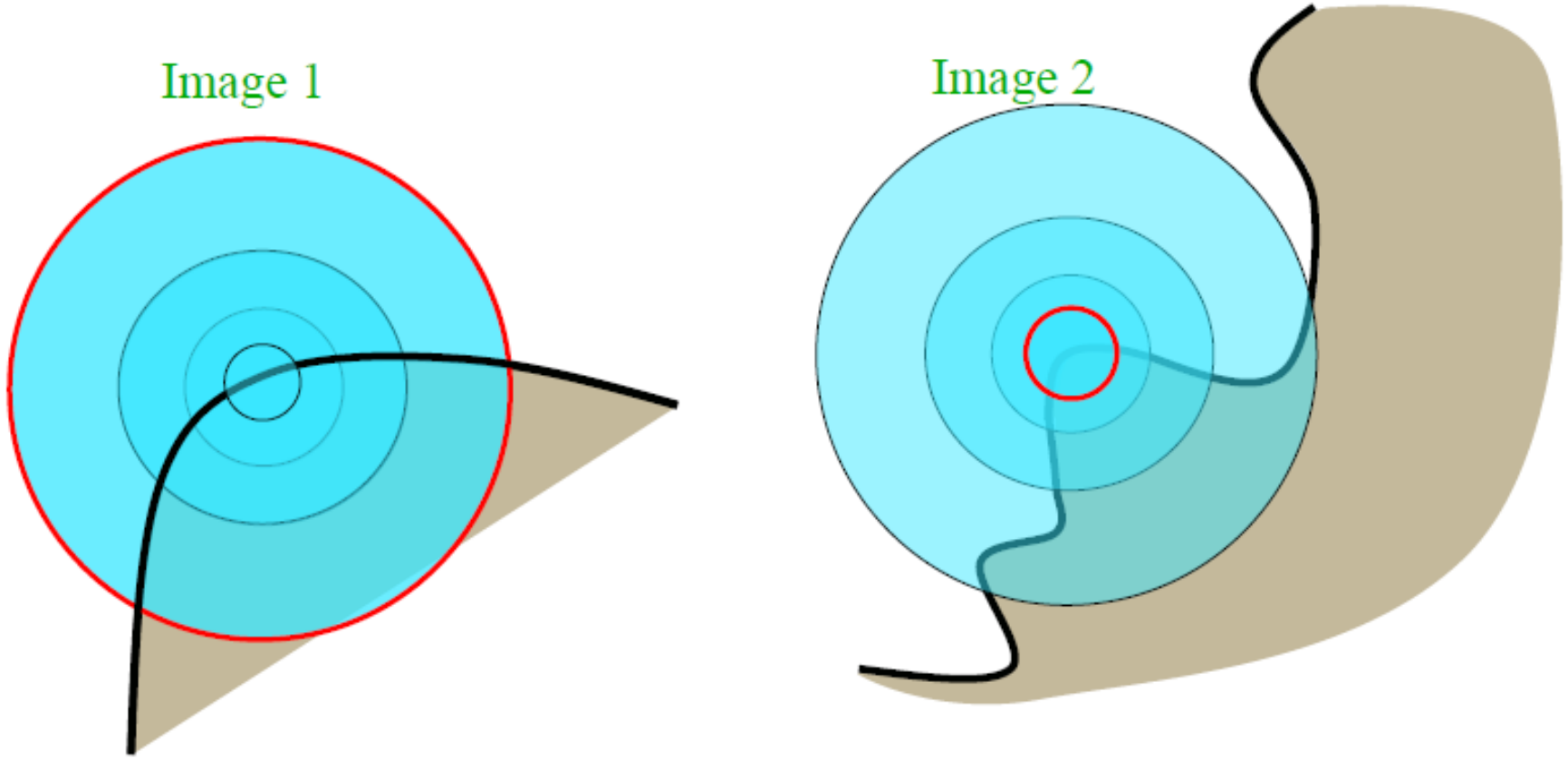
Самые важные свойства хорошего детектора признаков (после повторяемости **repeatability** и различимости **distinctiveness**) следующие:

- **Точность локализации:** Важно, чтобы обнаруженные признаки были точно определены как по положению на изображении, так и по масштабу.
- **Количество признаков:** Идеальное количество признаков зависит от применения. Для задач распознавания объектов или сцен, поиска изображений и 3D-реконструкции требуется большое количество признаков для увеличения точности. Для задач с семантической интерпретацией достаточно небольшого количества высокоуровневых признаков.
- **Инвариантность**: Признаки должны быть инвариантны к изменениям точки обзора камеры, освещения и масштаба. Это можно достичь с помощью математического моделирования преобразований. Примером успешной инвариантности является метод SIFT.
- **Вычислительная эффективность:** Важно, чтобы признаки могли быть обнаружены и сопоставлены быстро, особенно в реальном времени. Однако время вычислений зависит от уровня инвариантности: чем больше проверок нужно выполнить, тем дольше занимает процесс.
- **Устойчивость**: Обнаруженные признаки должны быть устойчивы к различным воздействиям, таким как шум, размытость, артефакты сжатия и отклонения от математической модели. Это обеспечивает надежность и точность в реальных условиях.

Scale-invariant feature detection (SIFT)

- Consider regions (e.g. discs) of different sizes around a point
- **Aim**: corresponding regions look the same in image space, when the appropriate scale-change is applied

Способ обнаружения угла в более высоком масштабе заключается в использовании многомасштабного детектора Харриса, применяя один и тот же детектор к изображению, каждый раз с другим размером окна (окружности).



- Choose corresponding regions (discs) **independently** in each image

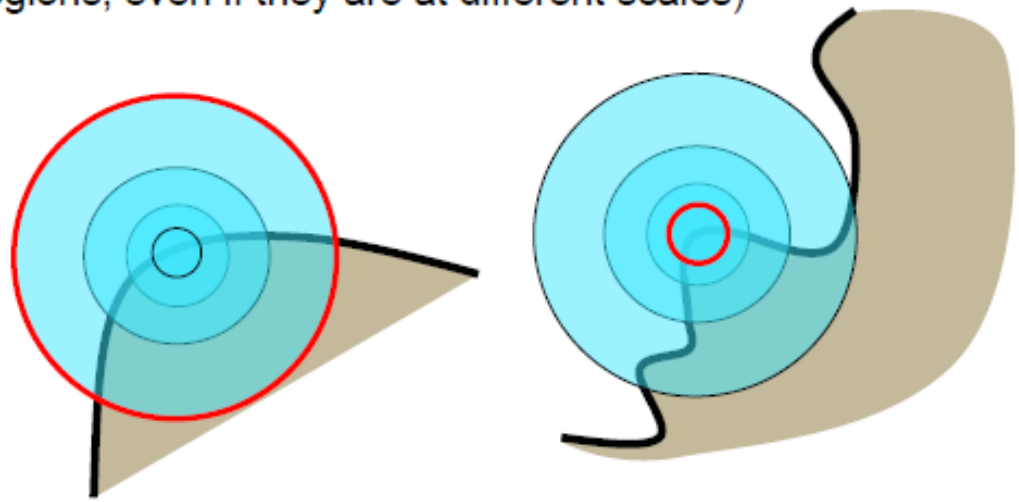
© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

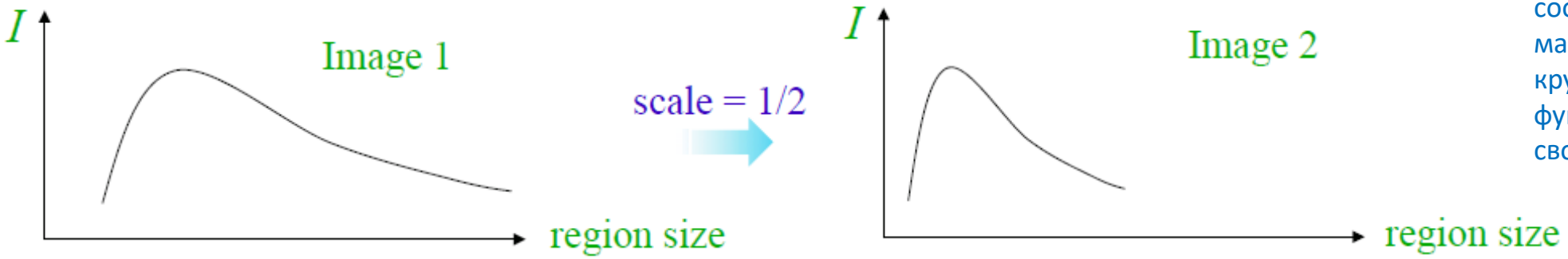
Scale-invariant feature detection

- Approach: design a function to apply on the region (disc) , which is “scale invariant”

(i.e. remains constant for corresponding regions, even if they are at different scales)

example: **average image intensity** over corresponding regions (at different scales) should remain constant





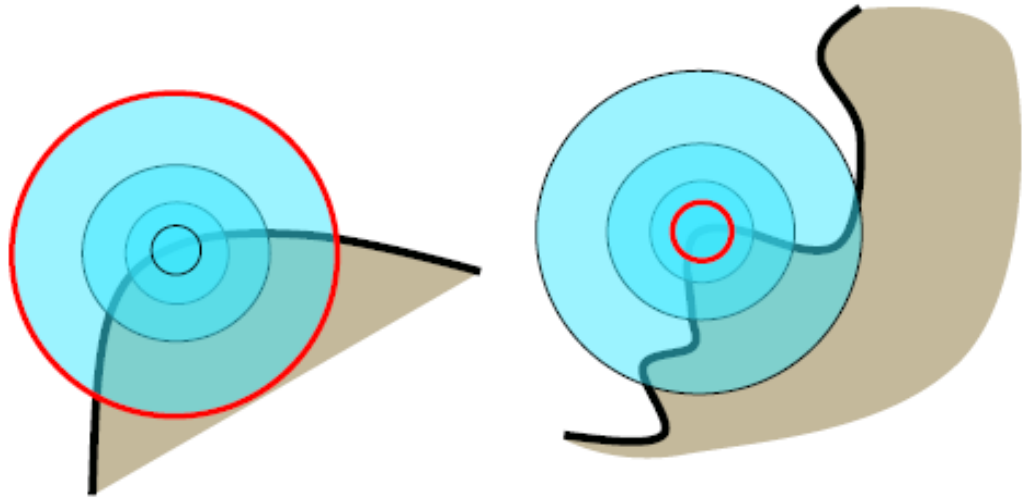
Для каждого круга на двух изображениях мы строим среднюю интенсивность пикселей внутри круга как функцию размера круга. Как и ожидалось, эти две функции выглядят одинаково вплоть до изменения масштаба по оси x. Решение состоит в том, чтобы принять в качестве соответствующих масштабов размеры кругов, при которых эти функции достигают своего максимума.

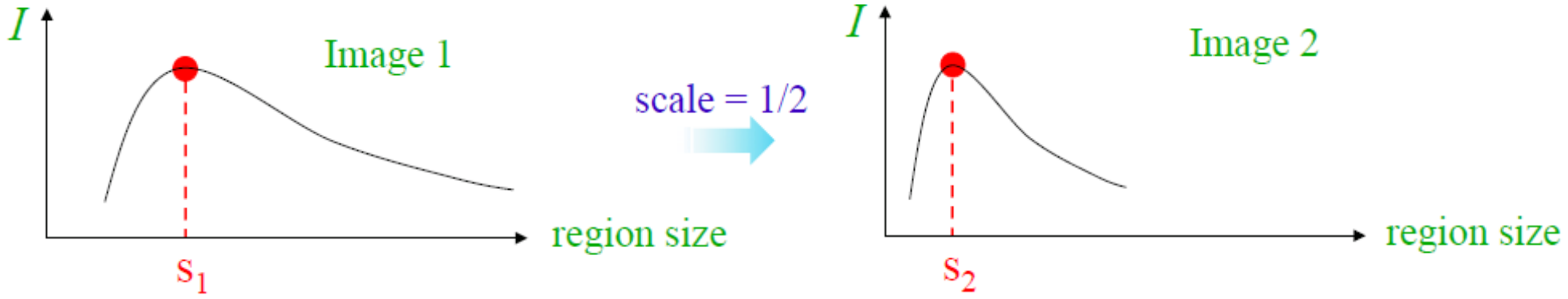
© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

Scale-invariant feature detection

- Identify the local maximum in each response ⇨ these occur at corresponding region sizes

The corresponding scale-invariant region size is found in each image independently!





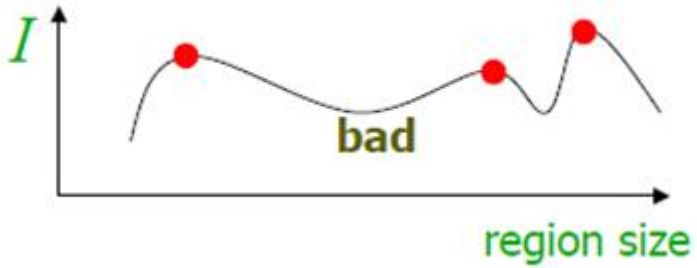
© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

Scale-invariant feature detection

- A “good” function for scale detection has one clear, sharp peak

Средняя интенсивность не годится, потому что она может возвращать несколько пиков или даже не иметь пиков вообще.







- Sharp, local intensity changes in an image, are good regions to monitor for identifying relative scale in usual images.

⇨ look for blobs or corners (i.e. sharp intensity discontinuities)

Собственно говоря, для обычных изображений хорошей функцией является та, которая соответствует контрасту, то есть резким локальным изменениям интенсивности. Лапласиан Гаусса (LoG) является хорошим оператором для определения резких изменений интенсивности и в настоящее время используется для выбора масштаба угловым детектором Харриса.

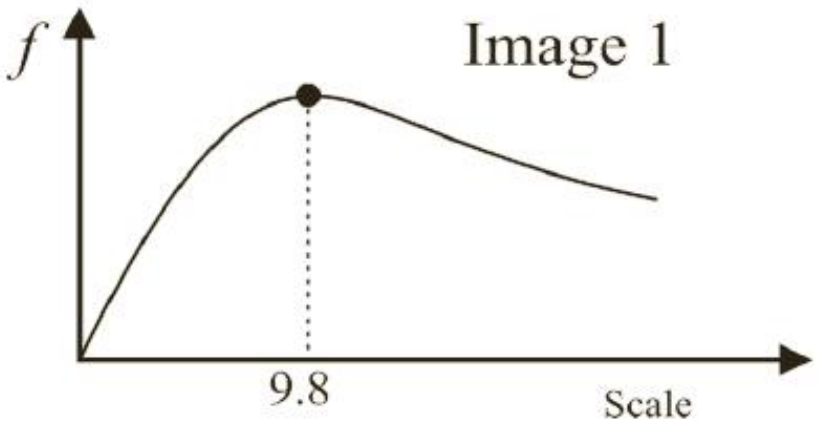
© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

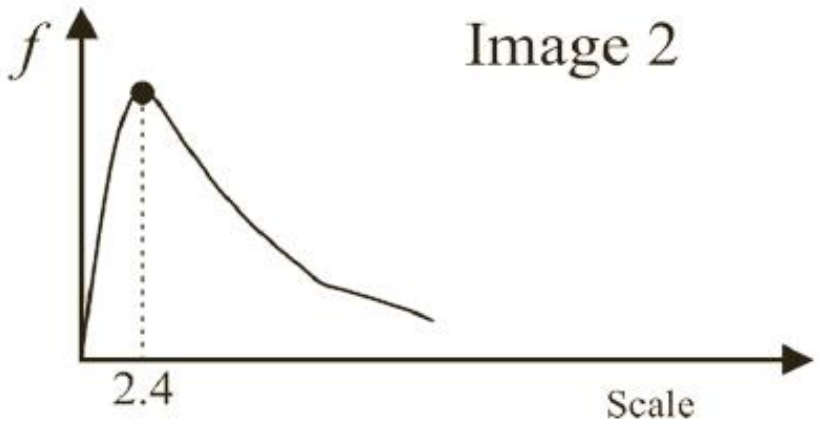
Scale-invariant feature detection | LoG scale detector

- Response of LoG for corresponding regions:









Оператор LoG показал один из лучших результатов по отношению к другим функциям выбора шкалы.

© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

Scale-invariant feature detection | LoG scale detector

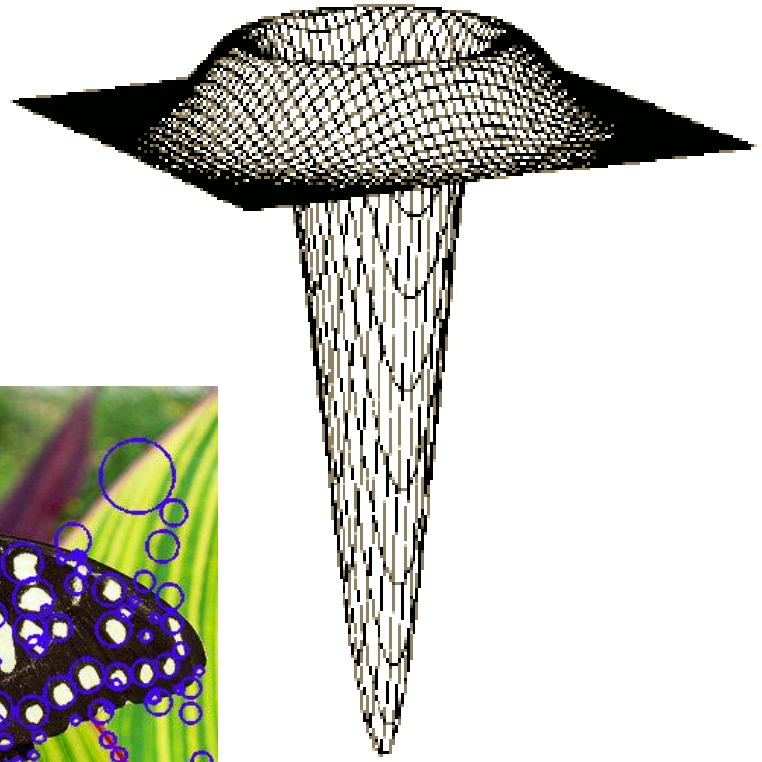
- Functions of determining scale: convolve image with kernel to identify sharp intensity discontinuities

$$f = \text{Kernel} * \text{Image}$$

- Detected scale corresponds to **local maxima or minima** of the convolved image region

$$LoG = \nabla^2 G(x,y) = \frac{\partial^2 G(x,y)}{\partial x^2} + \frac{\partial^2 G(x,y)}{\partial y^2}$$

Laplacian of Gaussian





Scale-invariant feature detection | LoG scale detector

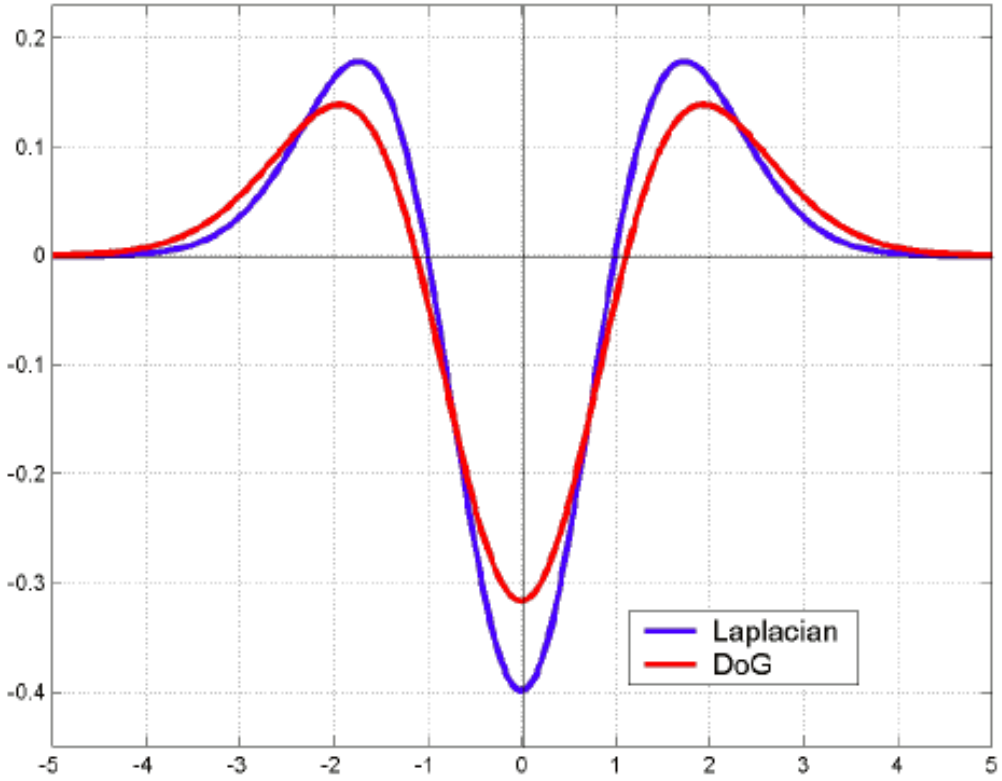
- Approximation to the LoG kernel for efficiency:

  Difference of Gaussians (DoG) kernel:

$$DoG = G_{k\sigma}(x,y) - G_{\sigma}(x,y)$$

- Used in the **SIFT** feature detector [Lowe et al., IJCV 2004]

- The **SURF** feature detector [Bay et al, CVIU 2008] implements the DoG kernel using a linear combination of rectangular functions



© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

SIFT features [Lowe et al., IJCV 2004]

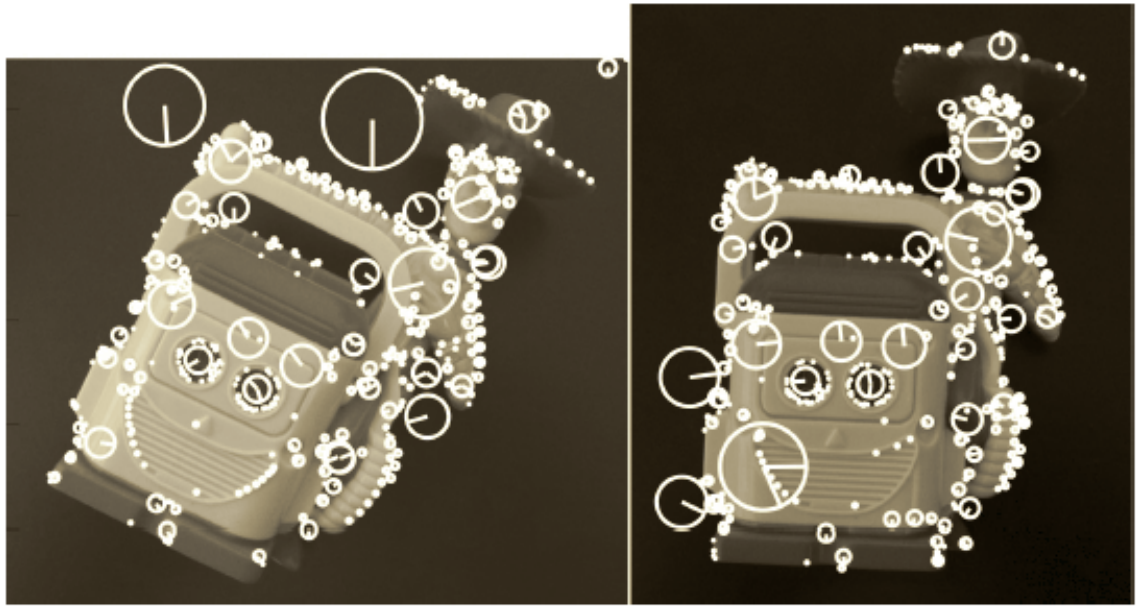
- **SIFT** = Scale Invariant Feature Transform
  an approach for detecting and describing regions of interest in an image
- SIFT features are reasonably **invariant** to changes in:
  rotation, scaling, changes in viewpoint, illumination
- SIFT **detector** uses **DoG kernel**, SIFT **descriptor** is based on **gradient orientations**
- Very powerful in capturing + describing **distinctive** structure, but also **computationally demanding**

SIFT — это метод обнаружения и сопоставления надежных ключевых точек, который был изобретен в 1999 году Лоу.

Его высокая повторяемость и высокая скорость совпадения в сложных условиях сделали SIFT одним из лучших детекторов признаков на сегодняшний день.

Main SIFT stages:

1. Extract keypoints + scale
2. Assign keypoint orientation
3. Generate keypoint descriptor

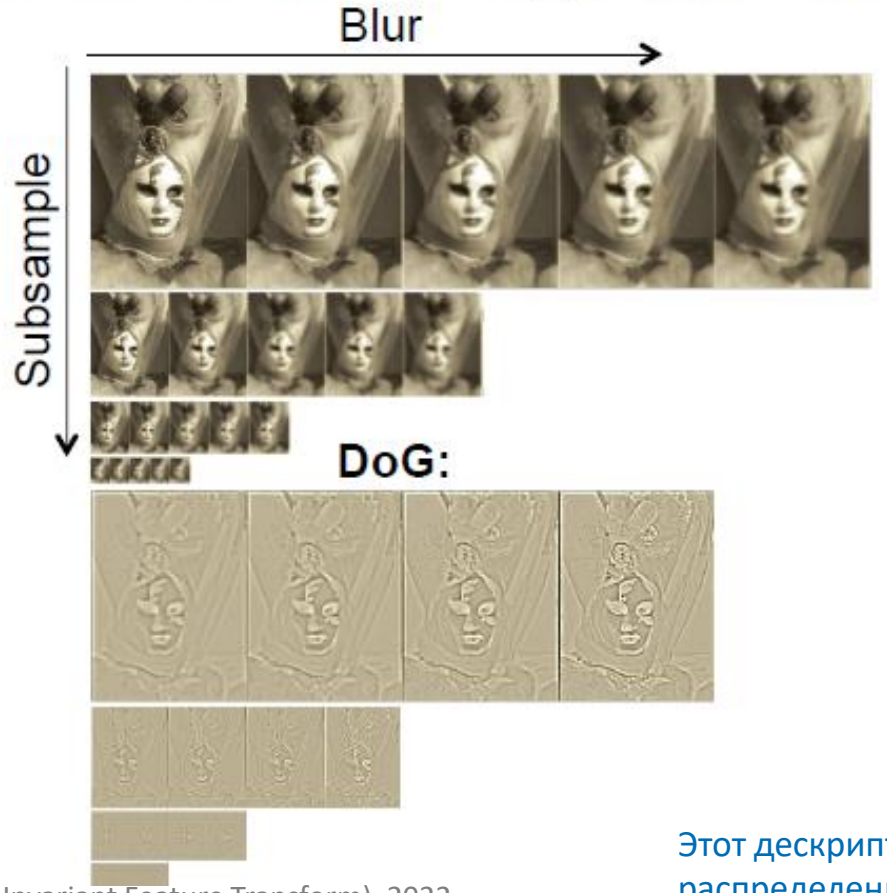


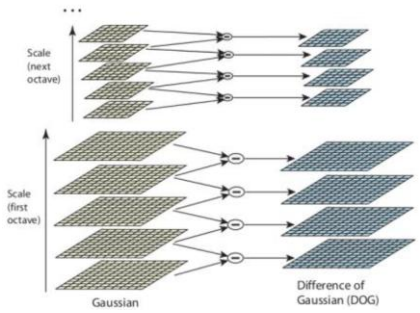
© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

SIFT features | detector (keypoint location + scale)

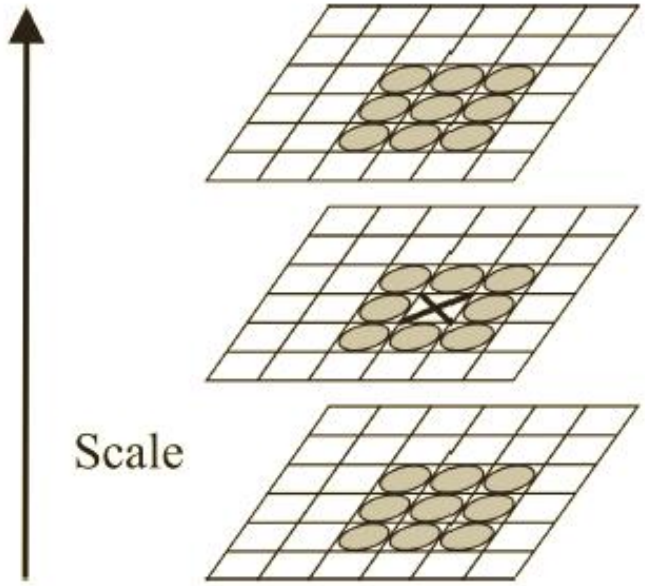
1. Scale-space pyramid: subsample and blur original image
2. Difference of Gaussians (DoG) pyramid: subtract successive smoothed images

Основное преимущество функций SIFT по сравнению со всеми другими методами заключается в том, что «дескриптор» вычисляется из области вокруг интересующей точки, которая отчетливо описывает информацию, переносимую функцией.





3. Keypoints: local extrema in the DoG pyramid



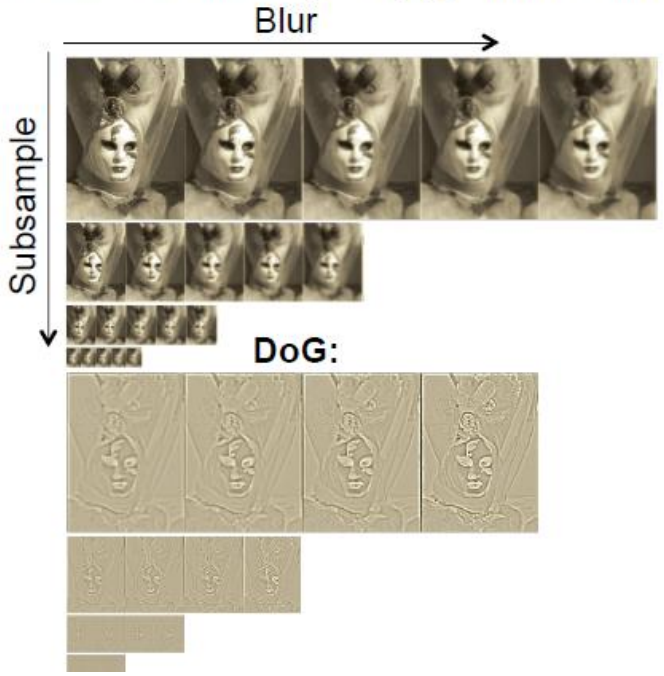
Этот дескриптор представляет собой вектор, представляющий локальное распределение градиентов изображения вокруг точки интереса.

Fig. © Introduction to SIFT (Scale-Invariant Feature Transform), 2022, https://docs.opencv.org/4.x/da/df5/tutorial_py_sift_intro.html

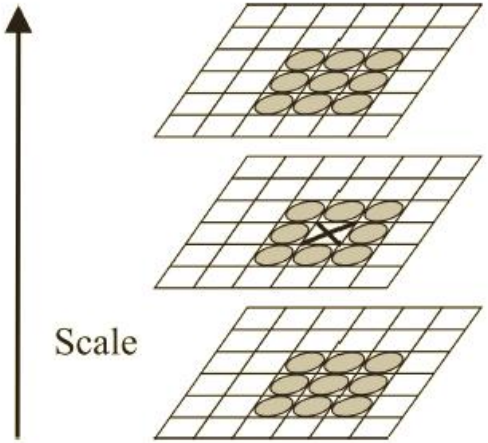
© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

SIFT features | detector (keypoint localization + scale)

1. Scale-space pyramid: subsample and blur original image
2. Difference of Gaussians (DoG) pyramid: subtract successive smoothed images



3. Keypoints: local extrema in the DoG pyramid



1-ый шаг к идентификации ключевых точек SIFT является создание так называемых изображений **разности Гаусса (DoG)**. Это делается путем первого размытия исходного изображения фильтрами Гаусса в разных масштабах (т. е. с разными сигмами), а затем путем взятия разности последовательных размытых по Гауссу изображений. Показан этот процесс: исходное изображение (вверху слева) размывается четырьмя фильтрами Гаусса с разными сигмами, и это повторяется после понижения дискретизации изображения с коэффициентом 2. Наконец, изображения DoG вычисляются путем простого определения разницы между последовательными размытыми изображениями.

2-й шаг – выбор ключевых точек. Ключевые точки SIFT идентифицируются как локальные максимумы или минимумы изображений DoG в разных масштабах. В частности, каждый пиксель в изображениях DoG сравнивается с восемью его соседями в том же масштабе плюс с девятью соседями в соседних масштабах. Если пиксель является локальным максимумом или минимумом, он выбирается в качестве ключевой точки-кандидата.

Ключевые точки с низким контрастом, нестабильные ключевые точки и ключевые точки, расположенные на краях, устраняются. DoG имеет более высокий отклик для краев, поэтому края также необходимо удалить. Для этого используется концепция, аналогичная детектору углов Харриса. Они использовали 2x2 матрицу Гессе (H) для вычисления главной кривизны. Из детектора углов Харриса мы знаем, что для краев одно собственное значение больше другого. Поэтому здесь они использовали простую функцию

© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

SIFT features | keypoint orientation assignment

Define “orientation” of keypoint to achieve **rotation invariance**

- Sample intensities around the keypoint
- Compute a histogram of orientations of intensity gradients

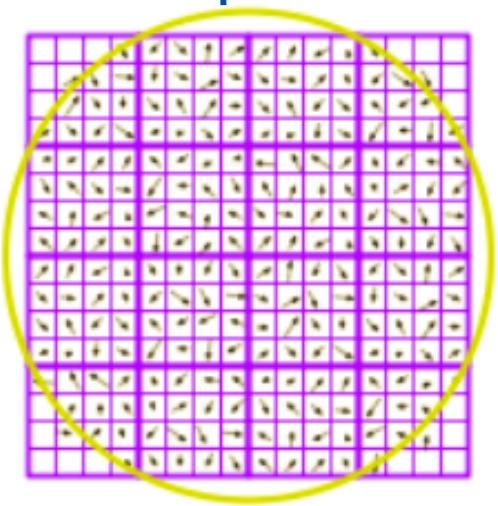
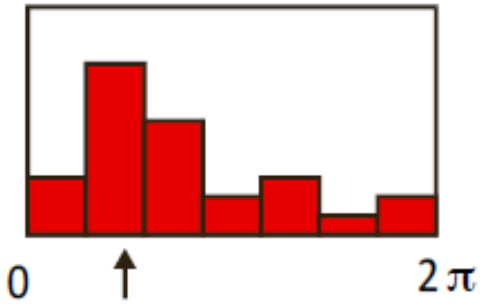


Image gradients

3-й шаг состоит в уточнении местоположения ключевых точек как в пространстве, так и в масштабе путем интерполяции близлежащих данных. Наконец, ключевые точки с низким контрастом или вдоль краев удаляются из-за их низкой различимости и из-за их неустойчивости к шуму изображения.

- Peaks in histogram: dominant orientations
- **Keypoint orientation = histogram peak**
- If there are multiple candidate peaks, construct a different keypoint for each such orientation

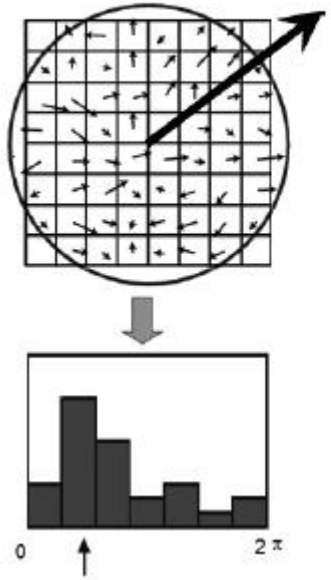


В то время как в Harris ключевая точка идентифицируется в плоскости изображения как локальный максимум функции угла, в SIFT ключевая точка является локальным минимумом или максимумом изображения DoG как в положении, так и в масштабе.

© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

SIFT features | Orientation assignment

Этот шаг состоит в назначении каждой ключевой точке определенной ориентации, чтобы сделать ее **инвариантной к вращению изображения**. Чтобы определить ориентацию ключевой точки, **гистограмма ориентации градиента** вычисляется в окрестности ключевой точки. Другими словами, для каждого пикселя в окрестностях ключевых точек вычисляется **градиент интенсивности** (**величина** и **ориентация**). Затем строится гистограмма ориентаций, в которой **вклад каждого пикселя взвешивается величиной градиента**. Пики на гистограмме соответствуют доминирующим ориентациям. После заполнения гистограммы **ключевой точке назначается ориентация, соответствующая самому высокому пику**. В случае **нескольких пиков**, которые находятся **в пределах 80%** от самого высокого пика, для каждой дополнительной ориентации создается **дополнительная характерная точка**, имеющая то же местоположение и масштаб, что и исходная характерная точка. Все свойства характерной точки будут измеряться относительно ориентации характерной точки. Это обеспечивает **инвариантность к вращению**. Окончательные характерные точки с выбранной ориентацией и масштабом показаны на рисунке.



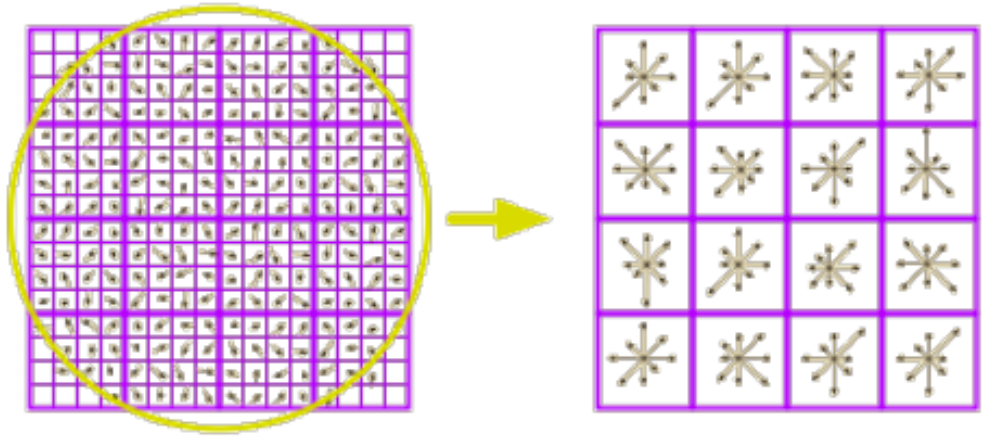


© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

SIFT features | descriptor

- **Descriptor** : “identity card” of keypoint
- Simplest descriptor: matrix of intensity values around a keypoint (image patch)
- Ideally, a descriptor should be
  - highly distinctive +
  - tolerant/invariant to common image transformations

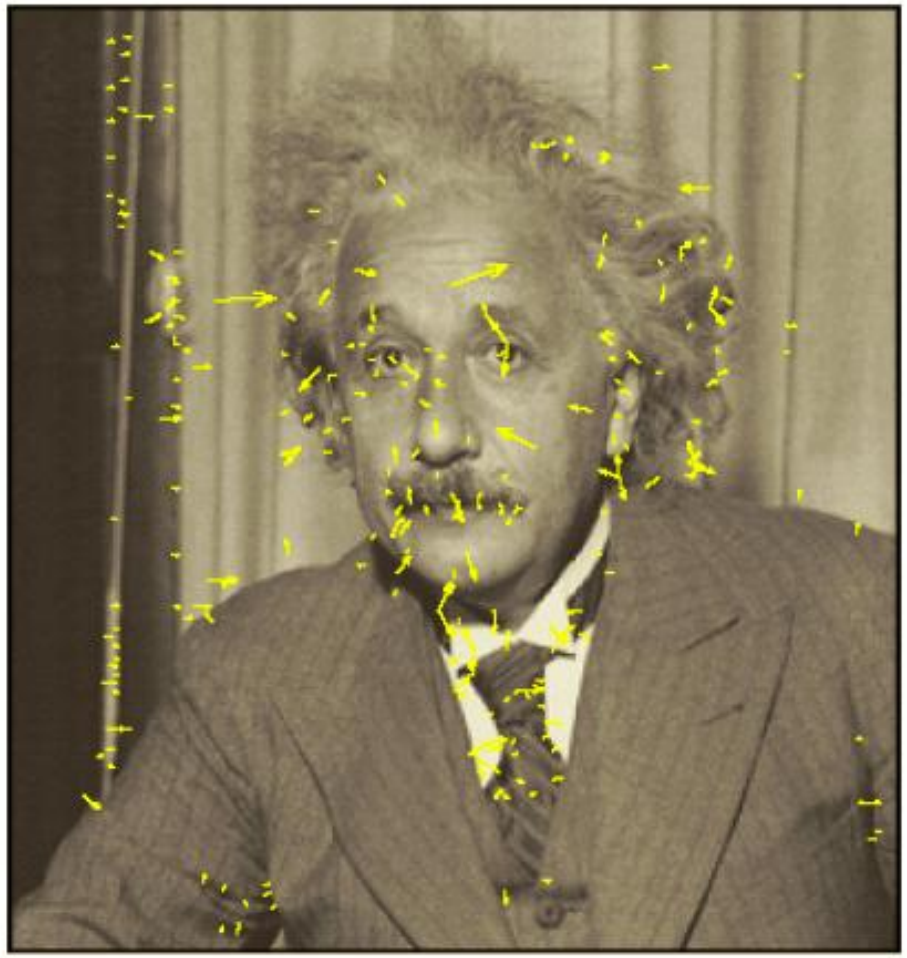
- **SIFT descriptor**: 128-element vector
- Describe all gradient orientations **relative to the keypoint orientation**
- Divide keypoint neighborhood in **4×4** regions and compute orientation histograms along **8** directions
- SIFT descriptor: concatenation of all **4×4×8 (=128)** values
- Descriptor Matching: L_2-distance (i.e. SSD) between these descriptor vectors



© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

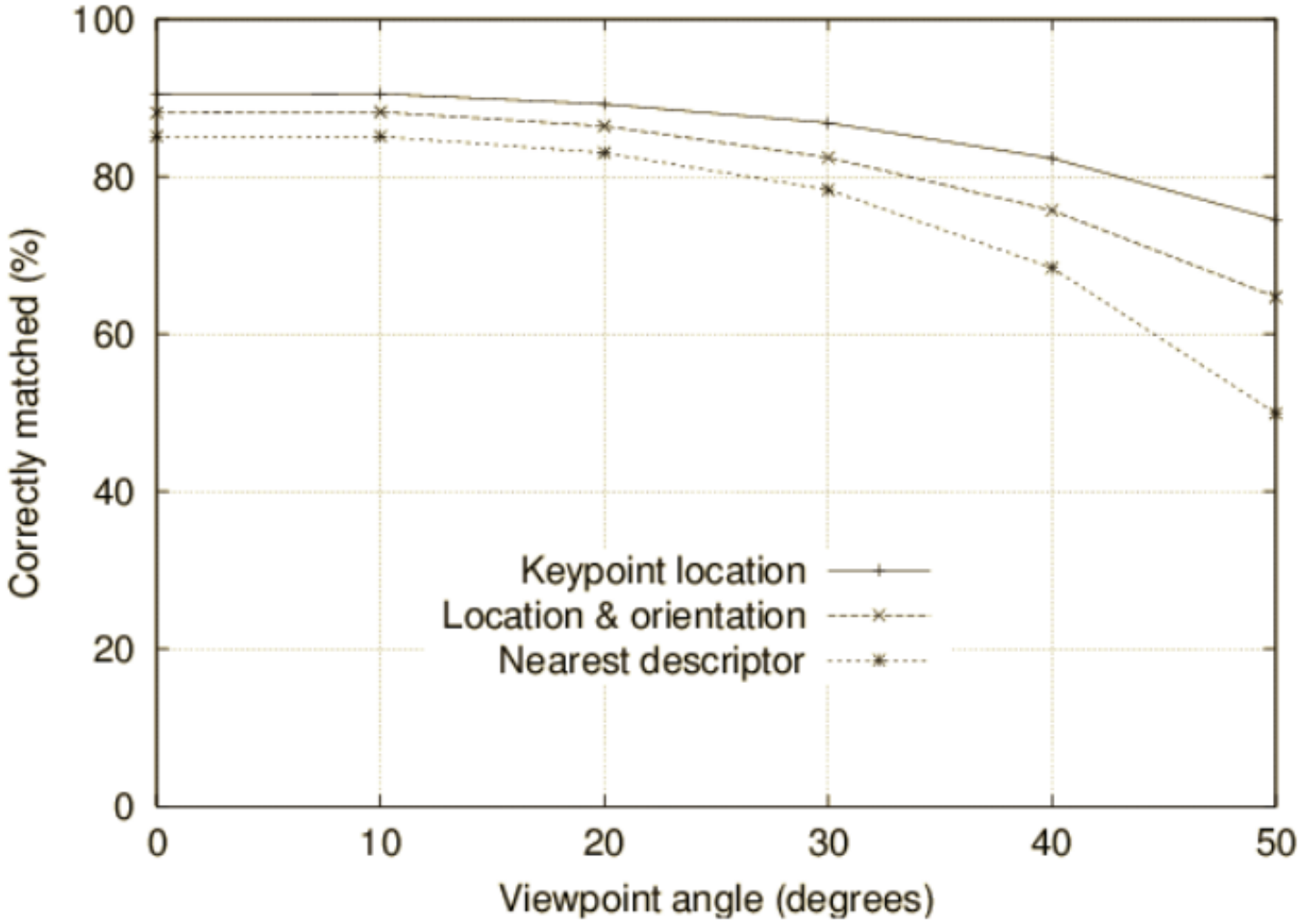
SIFT features | descriptor

- Final SIFT keypoints with detected orientation & scale



© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

SIFT features | features' stability to viewpoint change

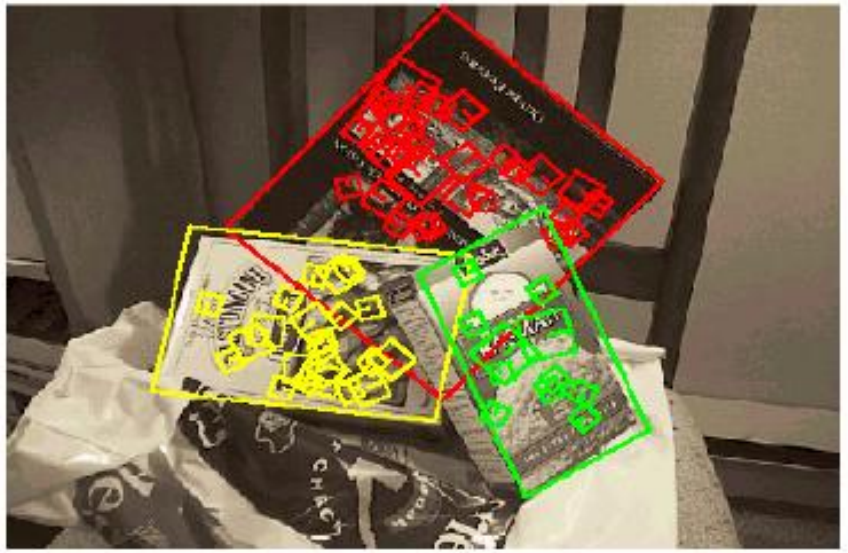


© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

SIFT features | use in planar recognition

- **Planar** surfaces can be reliably recognized at a rotation of **60°** away from the camera
- Only 3 points are needed for recognition
- But objects need to have enough **texture**
- Recognition under occlusion



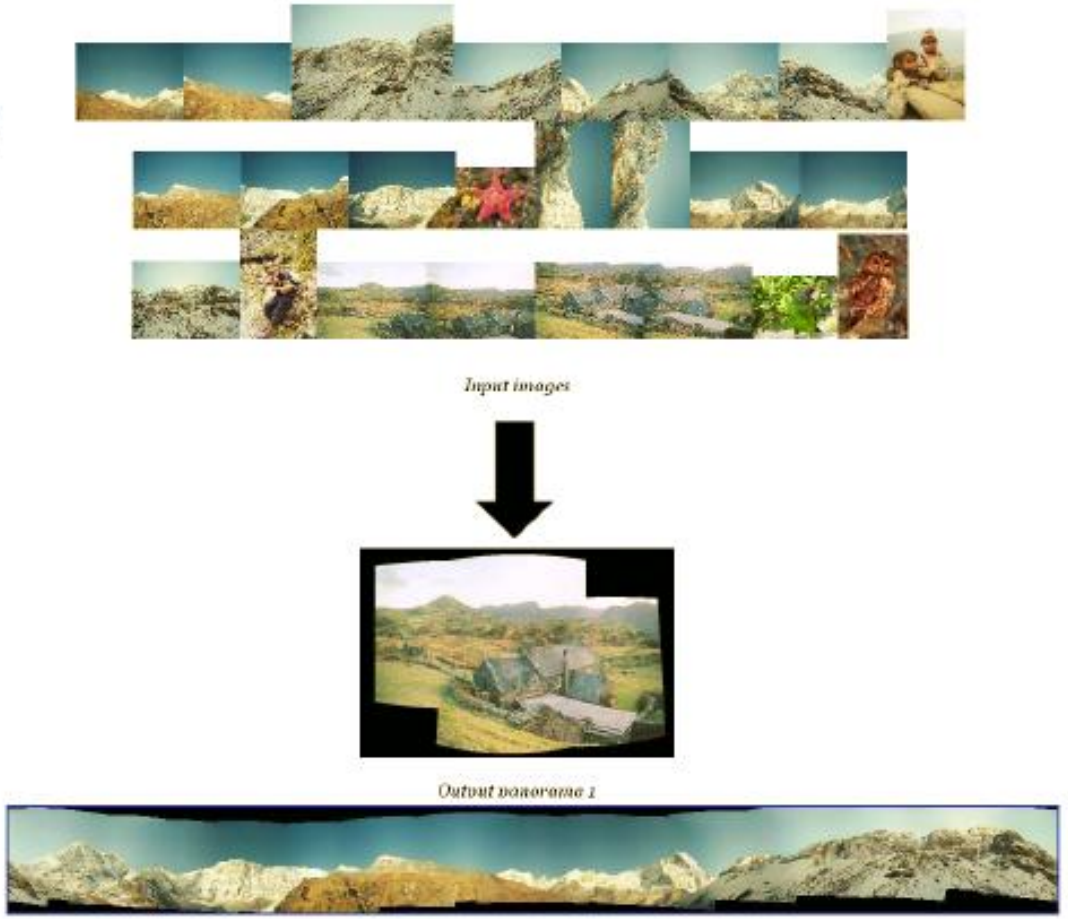


© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

SIFT features | code and demos

- **SIFT feature detector code:**
  for Matlab & C code to run with compiled binaries
  for Win and Linux (freeware)
  http://www.cs.ubc.ca/~lowe/keypoints/

- Make your own panorama with **AUTOSTITCH** (freeware):
  http://www.cs.ubc.ca/~mbrown/autostitch/autostitch.html



© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

SIFT in OpenCV

Function	Description
cv2.SIFT_create()	extracts keypoints and computes descriptors using the Scale Invariant Feature Transform (SIFT) algorithm by D. Lowe

```
import numpy as np
import cv2 as cv

img = cv.imread('home.jpg')
gray= cv.cvtColor(img,cv.COLOR_BGR2GRAY)

sift = cv.SIFT_create()
kp = sift.detect(gray,None)

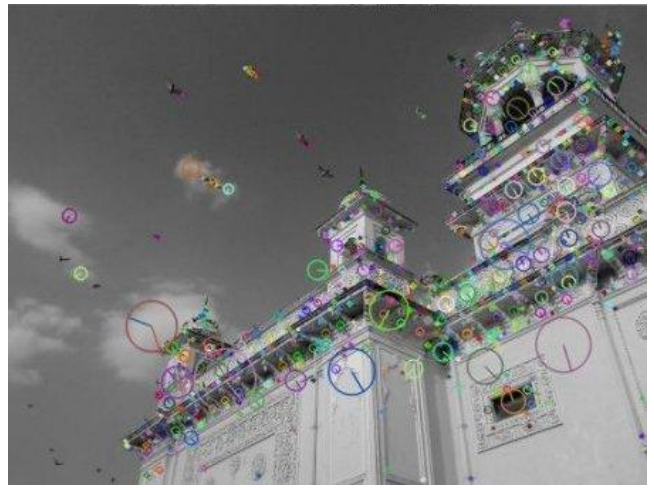
img=cv.drawKeypoints(gray,kp,img)

cv.imwrite('sift_keypoints.jpg',img)
```

cv2.SIFT_create() finds the keypoints in the image using the parameters:

- **nfeatures** - The number of best features to retain
- **nOctaveLayers** - The number of layers in each octave. 3 is the value used in D. Lowe paper. The number of octaves is computed automatically from the image resolution.
- **contrastThreshold** - The contrast threshold used to filter out weak features in semi-uniform (low-contrast) regions. The larger the threshold, the less features are produced by the detector.
- **edgeThreshold** - The threshold used to filter out edge-like features. Note that the its meaning is different from the contrastThreshold, i.e. the larger the edgeThreshold, the less features are filtered out (more features are retained).
- **sigma** - The sigma of the Gaussian applied to the input image at the octave #0

- sift.detect() function finds the keypoint in the images. You can pass a mask if you want to search only a part of image. Each keypoint is a special structure which has many attributes like its (x,y) coordinates, size of the meaningful neighbourhood, angle which specifies its orientation, response that specifies strength of keypoints etc.
- OpenCV also provides cv.drawKeyPoints() function which draws the small circles on the locations of keypoints. If you pass a flag, cv.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS to it, it will draw a circle with size of keypoint and it will even show its orientation.
- **sift.compute()** computes the descriptors from the keypoints we have found. Eg: kp,des = sift.compute(gray,kp)
- sift.detectAndCompute() directly find keypoints and descriptors in a single step
- kp will be a list of keypoints and des is a numpy array of shape (Number of Keypoints)×128.



© Introduction to SIFT (Scale-Invariant Feature Transform), OpenCV docs, 2022, https://docs.opencv.org/4.x/da/df5/tutorial_py_sift_intro.html

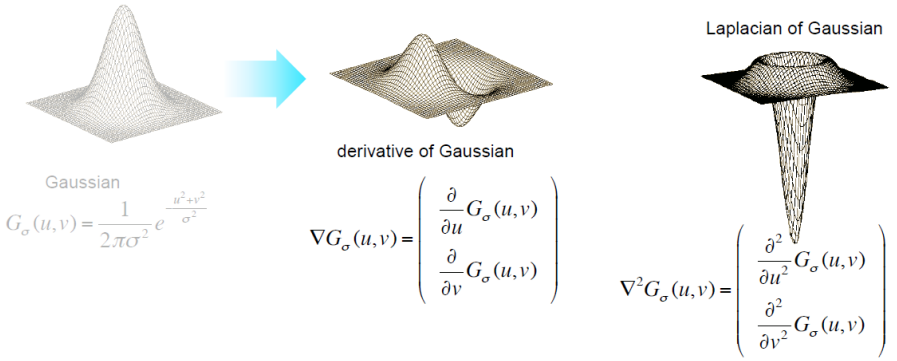
2

Обзор детекторов фитч в OpenCV

SURF · коробчатый фильтр (box filter) · матрица Гессе · детектор Harris-Affine · детектор FAST · бинарный дескриптор (BRIEF) · Алгоритм ORB (oFAST + rBRIEF) · Расстояние Хэмминга

SURF (Speeded Up Robust Features) in OpenCV

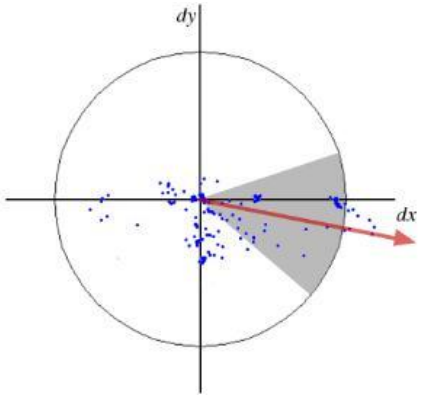
- **SIFT (Scale-Invariant Feature Transform)**: В SIFT Дэвид Лоу (David Lowe) использовал приближение Лапласиана Гауссиана (Laplacian of Gaussian) разницей Гауссианов (Difference of Gaussian) для поиска пространств масштабов (scale-space).
- **SURF (Speeded Up Robust Features)**: В SURF алгоритм идет немного дальше и приближает Лапласиан Гауссиана с помощью **коробчатого фильтра** (Box Filter).
- Одним из основных преимуществ такого приближения является то, что **свертка с коробчатым фильтром** может быть легко рассчитана с помощью **интегральных изображений**. И это может быть сделано параллельно для различных масштабов.
- Также алгоритм SURF полагается на **определитель матрицы Гессе** (determinant of Hessian matrix) как для масштаба, так и для определения местоположения.



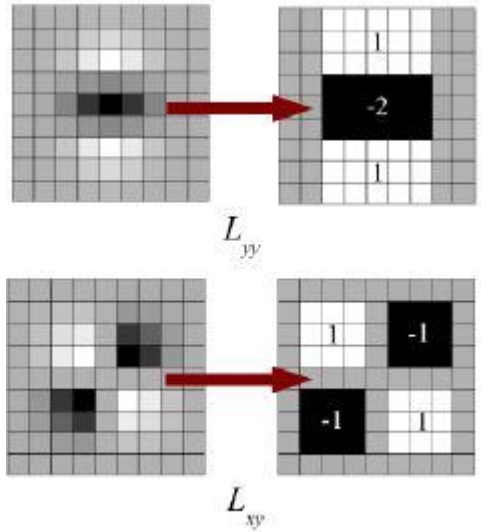
Hessian matrix in a point ***x*** *= (x,y)* at scale *σ:*

$$\mathcal{H}(\mathbf{x}, \sigma) = \begin{bmatrix} L_{xx}(\mathbf{x}, \sigma) & L_{xy}(\mathbf{x}, \sigma) \\ L_{xy}(\mathbf{x}, \sigma) & L_{yy}(\mathbf{x}, \sigma) \end{bmatrix}$$

- Для определения ориентации в SURF используются **отклики вейвлетов** (wavelet responses) в горизонтальном и вертикальном направлении для окрестности размером 60°.
- К ним также применяются соответствующие **гауссовские веса** (Gaussian weights). Затем они отображаются в пространстве, как показано на изображении ниже.
- **Доминирующая ориентация** оценивается путем вычисления суммы всех откликов **в скользящем окне ориентации** углом в 60 градусов.

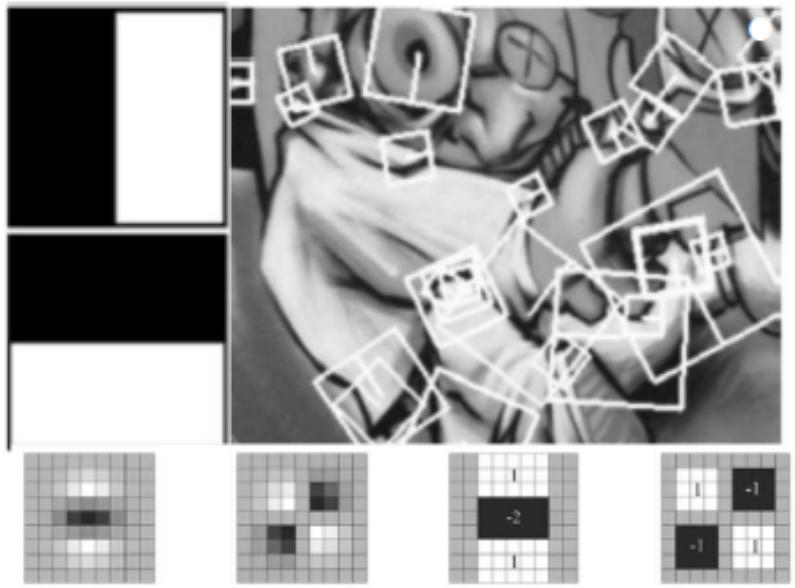


Детектор SURF:
В левой колонке представлены дискретизированные и обрезанные **вторые частные производные Гауссиана** по направлению y и по направлению xy. В правой колонке показаны **приближения Гауссианов с использованием ящичных фильтров**. Серые области равны нулю.



© Introduction to SURF (Speeded-Up Robust Features), OpenCV docs, 2022, https://docs.opencv.org/3.4/df/dd2/tutorial_py_surf_intro.html

Local Descriptors: SURF (Speeded Up Robust Features)



Fast approximation of SIFT idea

Efficient computation by 2D box filters & integral images
⇒ 6 times faster than SIFT

Equivalent quality for object identification

http://www.vision.ee.ethz.ch/~surf

http://www.vision.ee.ethz.ch/~surf

GPU implementation available

Feature extraction @ 100Hz
(detector + descriptor, 640×480 img)

http://homes.esat.kuleuven.be/~ncorneli/gpusurf/

[Bay, ECCV'06], [Cornelis, CVGPU'08]

© Slide credit: Fei-Fei Li

SURF (Speeded Up Robust Features) in OpenCV

Function	Description
cv.xfeatures2d.SURF_create()	extracts keypoints and computes descriptors using the Speeded Up Robust Features (SURF) algorithm by Bay, H., Tuytelaars, T. and Van Gool, L. (2006)

SURF in OpenCV

OpenCV provides SURF functionalities just like SIFT. You initiate a SURF object with some optional conditions like 64/128-dim descriptors, Upright/Normal SURF etc. All the details are well explained in docs. Then as we did in SIFT, we can use SURF.detect(), SURF.compute() etc for finding keypoints and descriptors.

First we will see a simple demo on how to find SURF keypoints and descriptors and draw it. All examples are shown in Python terminal since it is just same as SIFT only.

```
>>> img = cv.imread('fly.png',0)

# Create SURF object. You can specify params here or later.
# Here I set Hessian Threshold to 400
>>> surf = cv.xfeatures2d.SURF_create(400)

# Find keypoints and descriptors directly
>>> kp, des = surf.detectAndCompute(img,None)

>>> len(kp)
 699
```

1199 keypoints is too much to show in a picture. We reduce it to some 50 to draw it on an image. While matching, we may need all those features, but not now. So we increase the Hessian Threshold.

```
# Check present Hessian threshold
>>> print( surf.getHessianThreshold() )
400.0

# We set it to some 50000. Remember, it is just for representing in picture.
# In actual cases, it is better to have a value 300-500
>>> surf.setHessianThreshold(50000)

# Again compute keypoints and check its number.
>>> kp, des = surf.detectAndCompute(img,None)

>>> print( len(kp) )
47
```

It is less than 50. Let's draw it on the image.

```
>>> img2 = cv.drawKeypoints(img,kp,None,(255,0,0),4)

>>> plt.imshow(img2),plt.show()
```

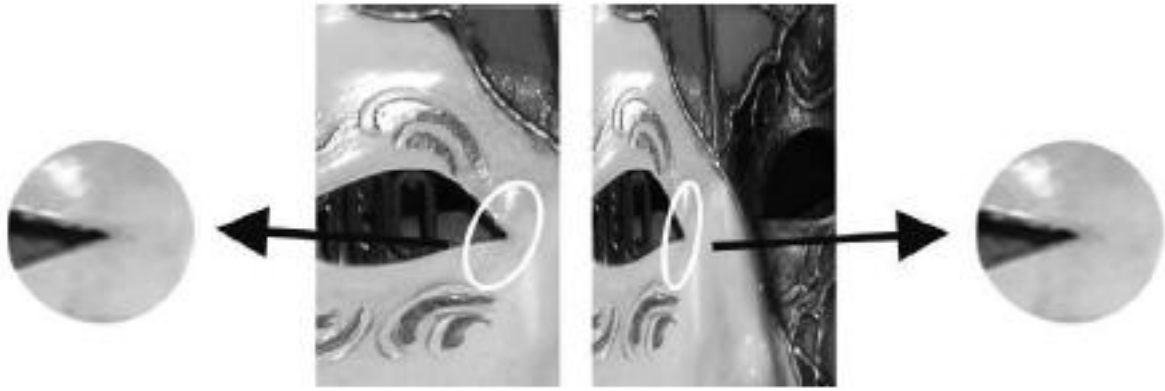
See the result below. You can see that SURF is more like a blob detector. It detects the white blobs on wings of butterfly. You can test it with other images.



© Introduction to SURF (Speeded-Up Robust Features), OpenCV docs, 2022, https://docs.opencv.org/3.4/df/dd2/tutorial_py_surf_intro.html

Affine invariant detection (Harris-Affine)

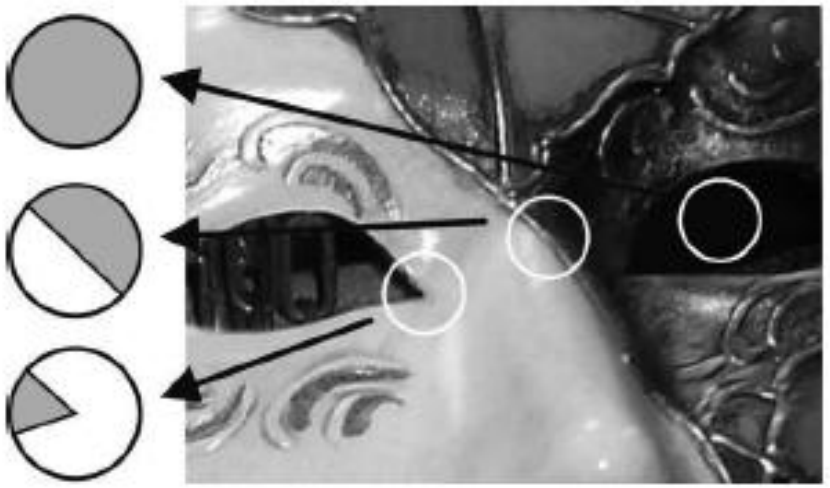
- **Афинное преобразование** является хорошим приближением **перспективного искажения** локально плоских областей при небольших изменениях точки наблюдения камеры.
- Как обнаружить те же самые признаки при аффинном преобразовании, которое можно рассматривать как **неравномерное масштабирование**?
- **Процедура** состоит из следующих шагов:
  1. Признаки идентифицируются с помощью **детектора Харриса-Лапласиана** (Harris-Laplacian), **инвариантного к масштабу**.
  2. **Матрица второго момента** используется для определения **двух направлений самых медленных и самых быстрых изменений интенсивности** вокруг признака.
  3. Из этих двух направлений **вычисляется эллипс**, размер которого соответствует масштабу, вычисленному с помощью **оператора Лапласиана Гаусса** (LoG).
  4. Область внутри **эллипса нормализуется до круглой формы**.
  5. Изначально обнаруженный **эллипс** и результирующая **нормализованная круглая форма** имеют **те же признаки**.



Вычисление **аффинно-инвариантного эллипса** в двух изображениях, связанных аффинным преобразованием

SUSAN corner detector

- **Smallest Univalue Segment Assimilating Nucleus** (SUSAN)
- **SUSAN** помимо использования для **обнаружения углов**, также используется для **обнаружения краев** и **подавления шума**.
- В отличие от Харриса, который основан на локальных градиентах изображения и вычислительно дорог, **SUSAN основан на морфологическом подходе**, который вычислительно более эффективен, чем Харрис.
- **Процедура** состоит из следующих шагов:
  1. Для каждого пикселя на изображении SUSAN рассматривает **круговое окно фиксированного радиуса с центром на** этом **пикселе**.
  2. Затем все пиксели внутри этого окна делятся на две категории, в зависимости от того, имеют ли они "похожие" или "разные" значения интенсивности по сравнению с центральным пикселем.
  3. На областях изображения с однородной интенсивностью большинство пикселей внутри окна будет иметь схожую яркость с центральным пикселем.
  4. **Возле краев доля пикселей** с похожей интенсивностью упадет до **50%**, **возле углов** она снизится еще больше до **около 25%**.
  5. В заключительном шаге используется **подавление немаксимумов** для определения **локальных минимумов**.



SUSAN corners

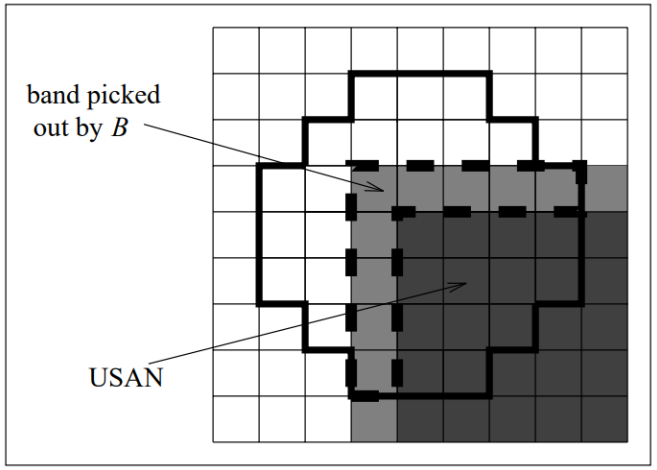
Углы **SUSAN** показывают **высокую повторяемость**, однако они сильно **чувствительны к большому шуму**. Кроме того, **многие признаки часто расположены на краях**, **а не на** реальных **углах**.

SUSAN corner detector

- Самый малый унифицированный сегмент, ассимилирующий ядро (SUSAN):
  Смит и Брэйди (IJCV, 1997) представили иной подход к обнаружению признаков в изображениях 1D и 2D, таких как края и углы. Алгоритм обнаружения углов **SUSAN не требует вычисления производных**, поэтому он хорошо работает с шумными изображениями. Основная идея **SUSAN** заключается в **использовании маски для подсчета количества пикселей**, имеющих **такую же яркость, как центральный пиксель**. Сравнивая количество пикселей, имеющих такую же яркость, как центральный пиксель, с порогом, детектор может определить, является ли центральный пиксель углом.

- Circular mask M consisting of 37 pixels should be used. The central pixel of the mask is called a nucleus. Then intensities of all pixels within a mask are compared with an intensity of a nucleus and an area of “similar” pixels is marked. This area is called USAN (Univalue Segment Assimilating Nucleus) and it conveys the most important information on a local structure of an image. Analyzing the size, centroid and the second moments of USAN the exact information on a type of local structure around a nucleus is inferred, such as edges or corners. For those regions, inverted USAN area shows strong peaks –thus the term SUSAN – i.e. the smallest USAN. This approach has an additional advantage of not using any derivatives which are cumbersome to use in the presence of noise.

- **Алгоритм SUSAN**:
  - Разместите **круговую маску вокруг пикселя** (т.е. ядра).
  - Подсчитайте **количество пикселей внутри круговой маски**, которые имеют схожую яркость с ядром. Такие пиксели составляют **USAN**.
  - Вычислите **силу угла** (определяется по числу похожих пикселей: чем меньше пикселей имеют схожую яркость, тем сильнее выражен угол).
  - Используйте **подавление немаксимумов для нахождения углов**.

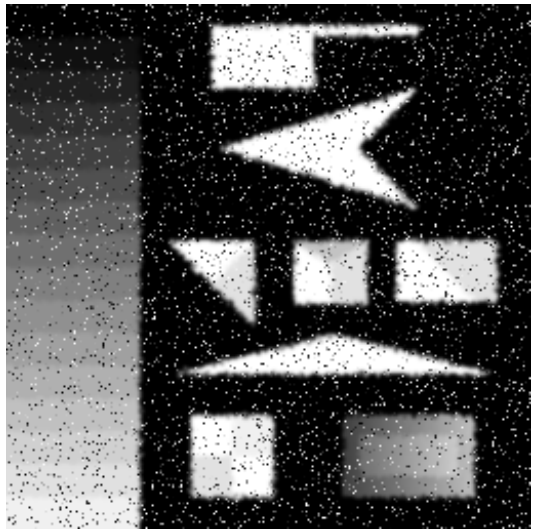


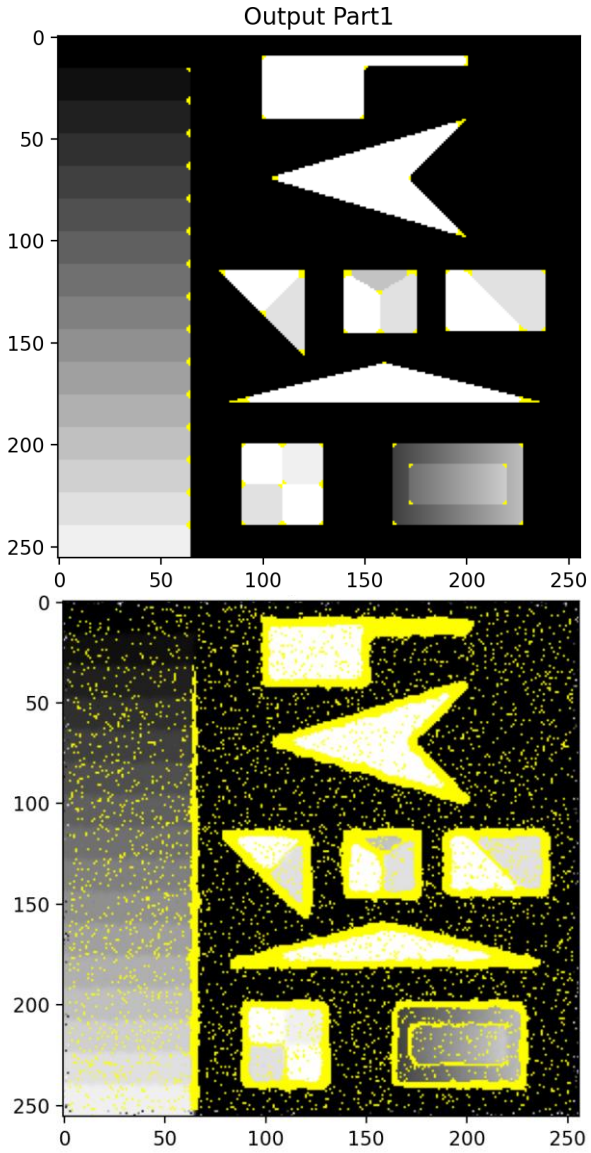
SUSAN corner (within 37 pixels of the circle)

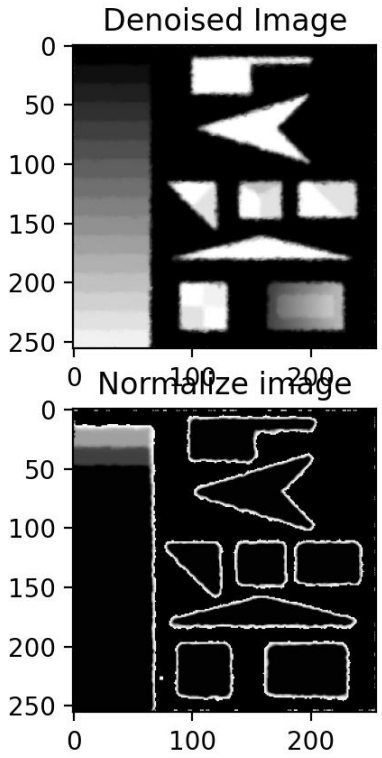
© Susan-Corner-Detection, 2017, https://github.com/rajatjain3571/Susan-Corner-Detection

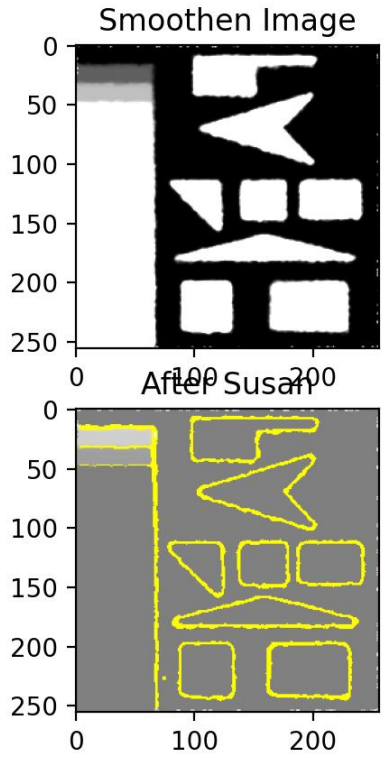
SUSAN corner detector











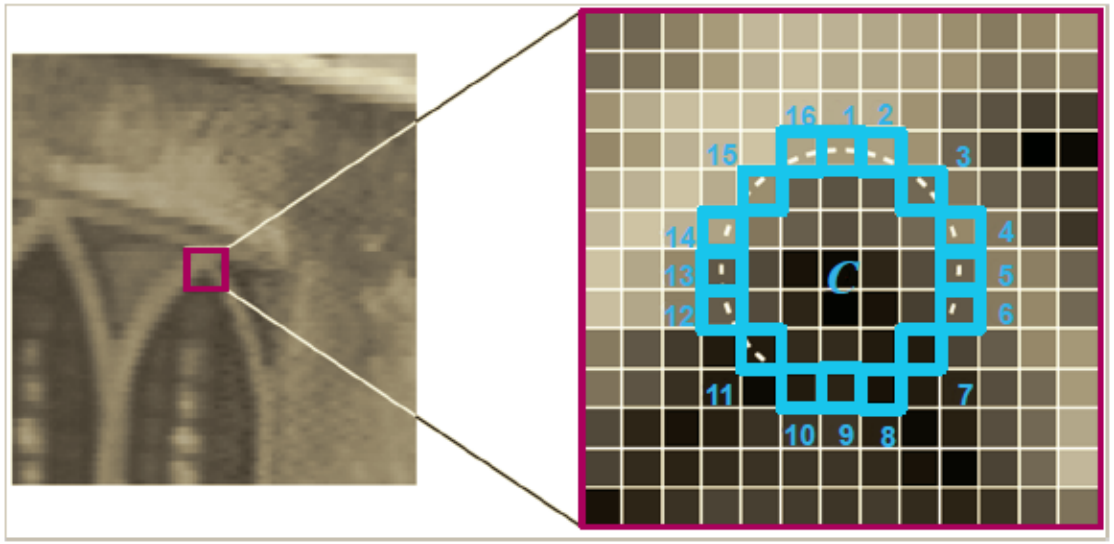
Since SUSAN corner detector is not robust to noise, the image should be denoised before using SUSAN detector

© Susan-Corner-Detection, 2017, https://github.com/rajatjain3571/Susan-Corner-Detection

FAST corner detector [Rosten et al., PAMI 2010]

- **FAST**: **F**eatures from **A**ccelerated **S**egment **T**est
- Studies intensity of pixels on circle around candidate pixel C

FAST is based on SUSAN detector



- C is a FAST corner **if** a set of N contiguous pixels on circle are:
  - all brighter than `intensity_of(C)+theshold`, or
  - all darker than `intensity_of(C)+theshold`
- Typical FAST mask: test for **12** contiguous pixels in a **16**-pixel circle
- **Very fast detector** – in the order of 100 Mega-pixel/second

FAST is up to 30 times faster than Harris

В то время как алгоритм SUSAN вычисляет долю пикселей внутри кругового окна, которые имеют схожую интенсивность с центральным пикселем, алгоритм **FAST сравнивает пиксели только на окружности** из 16 пикселей вокруг кандидата на угол.

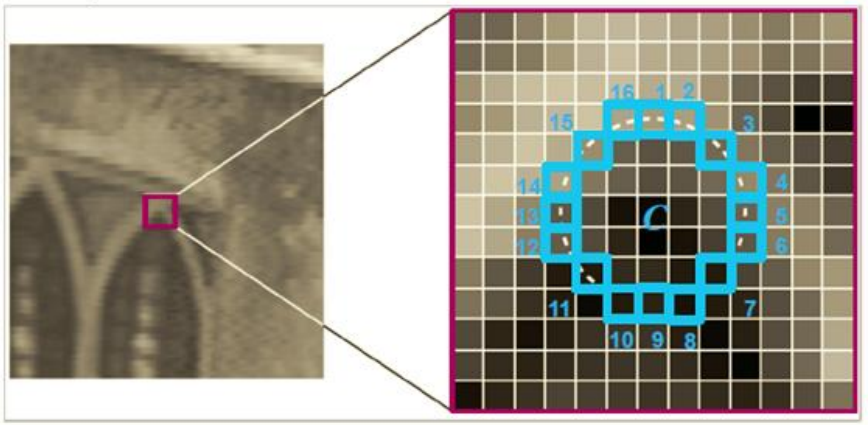
© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

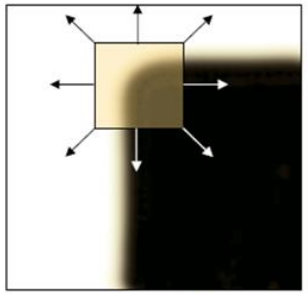
FAST (Features from Accelerated Segment Test) Detector

- Алгоритм **FAST (Features from Accelerated Segment Test)** был предложен Эдвардом Ростеном и Томом Драммондом в статье "Машинное обучение для высокоскоростного обнаружения углов" в 2006 году
- Поскольку мобильные роботы обладают ограниченными вычислительными ресурсами для SLAM, некоторые детекторы признаков не работают в реальном времени. FAST был разработан для решения этой задачи.

- Feature Detection using FAST

1. Выберите пиксель на изображении ***p*** с яркостью I_p.
2. Выберите подходящее пороговое значение ***t***.
3. Рассмотрите окружность из 16 пикселей вокруг тестируемого пикселя.
4. Пиксель ***p*** является углом, если существует набор из ***n*** смежных пикселей в окружности (из 16 пикселей), которые все ярче, чем I_p+t или все темнее, чем I_p-t. (Показаны белыми пунктирными линиями на изображении). Значение n было выбрано равным 12.
5. Для исключения большого количества неугловых пикселей используется тест, который проверяет только четыре пикселя на позициях 1, 9, 5 и 13 (сначала проверяются 1 и 9, если они слишком яркие или темные, затем проверяются 5 и 13). Если ***p*** является углом, то по крайней мере три из этих пикселей должны быть ярче, чем I_p+t или темнее, чем I_p-t. Если ни одно из этих условий не выполняется, то ***p*** не может быть углом.





"corner":
significant change in at least 2 directions

Детектор имеет высокую производительность, но у него есть слабые стороны:

- Когда n < 12, алгоритм менее эффективен в отбраковке неугловых пикселей (**ложные срабатывания**).
- **Порядок проверки пикселей** и распределение появления углов могут сильно **влиять на эффективность алгоритма**.
- Результаты высокоскоростного теста (для быстрого исключения неугловых пикселей) **не сохраняются**.
- Алгоритм может выявлять множество признаков (углов) вблизи друг друга, что может привести к **избыточности**.

© FAST Algorithm for Corner Detection, OpenCV docs, 2022, https://docs.opencv.org/3.4/df/d0c/tutorial_py_fast.html

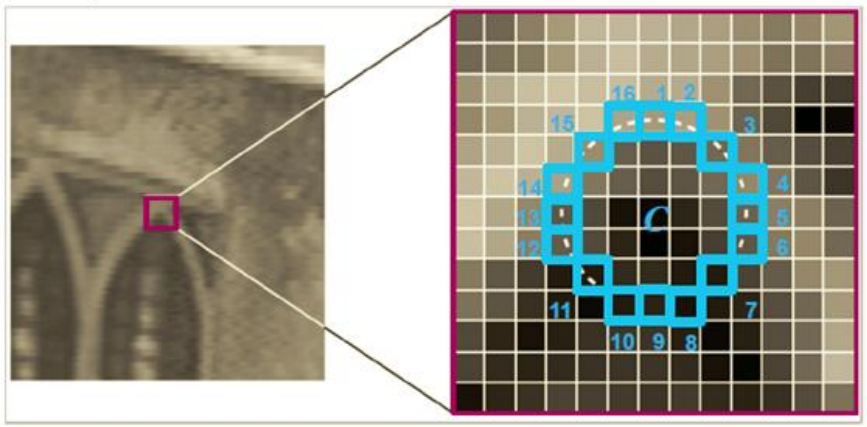
FAST (Features from Accelerated Segment Test) Detector

Machine Learning as a Corner Detector

Дерево решений (ID3): Чтобы не подбирать правила вручную, авторы использовали машинное обучение.

1. Select a set of images for training (preferably from the target application domain)
2. Run FAST algorithm in every images to find feature points.
3. For every feature point, store the 16 pixels around it as a vector. Do it for all the images to get feature vector *P*.
4. Each pixel (say *x*) in these 16 pixels can have one of the following three states:

$$S_{p\to x} = \begin{cases} d, & I_{p\to x} \le I_p - t & \text{(darker)} \\ s, & I_p - t < I_{p\to x} < I_p + t & \text{(similar)} \\ b, & I_p + t \le I_{p\to x} & \text{(brighter)} \end{cases}$$



5. Depending on these states, the feature vector P is subdivided into 3 subsets, P_d, P_s, P_b.
6. Define a new boolean variable, K_p, which is true if *p* is a corner and false otherwise.
7. Use the ID3 algorithm (decision tree classifier) to query each subset using the variable K_p for the knowledge about the true class. It selects the x which yields the most information about whether the candidate pixel is a corner, measured by the entropy of K_p.
8. This is recursively applied to all the subsets until its entropy is zero.
9. The decision tree so created is used for fast detection in other images.

Non-maximal Suppression

Detecting multiple interest points in adjacent locations is another problem. It is solved by using Non-maximum Suppression.

1. Compute a score function, *V* for all the detected feature points. *V* is the sum of absolute difference between *p* and 16 surrounding pixels values.
2. Consider two adjacent keypoints and compute their *V* values.
3. Discard the one with lower *V* value.

© FAST Algorithm for Corner Detection, OpenCV docs, 2022, https://docs.opencv.org/3.4/df/d0c/tutorial_py_fast.html

FAST corner detector in OpenCV

Function	Description
cv2.FastFeatureDetector_create()	initiates FAST object with default values
fast.detect()	detects keypoints with FAST corner detector

```
import numpy as np
import cv2 as cv
from matplotlib import pyplot as plt

img = cv.imread('blox.jpg',0) # `<opencv_root>/samples/data/blox.jpg`

# Initiate FAST object with default values
fast = cv.FastFeatureDetector_create()

# find and draw the keypoints
kp = fast.detect(img,None)
img2 = cv.drawKeypoints(img, kp, None, color=(255,0,0))

# Print all default params
print( "Threshold: {}".format(fast.getThreshold()) )
print( "nonmaxSuppression:{}".format(fast.getNonmaxSuppression()) )
print( "neighborhood: {}".format(fast.getType()) )
print( "Total Keypoints with nonmaxSuppression: {}".format(len(kp)) )

cv.imwrite('fast_true.png', img2)

# Disable nonmaxSuppression
fast.setNonmaxSuppression(0)
kp = fast.detect(img, None)

print( "Total Keypoints without nonmaxSuppression: {}".format(len(kp)) )

img3 = cv.drawKeypoints(img, kp, None, color=(255,0,0))

cv.imwrite('fast_false.png', img3)
```

cv2.FastFeatureDetector_create parameters:

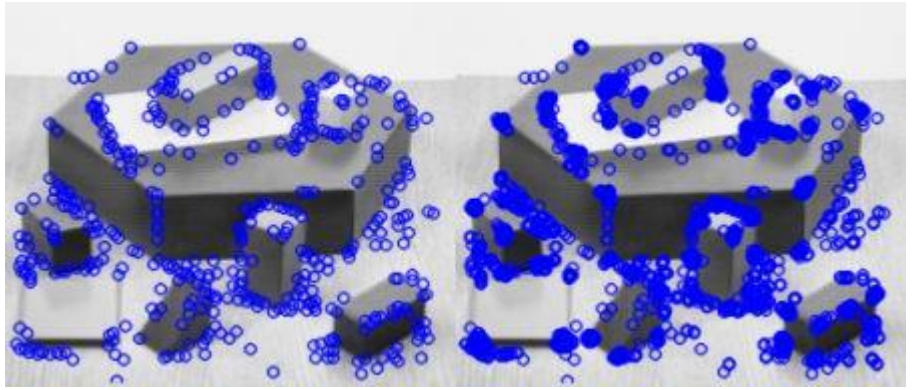
- **Threshold** – intensity threshold value
- **nonmaxSuppression:True** or **False**
- **Neighborhood** - For the neighborhood, three flags are defined, cv.FAST_FEATURE_DETECTOR_TYPE_5_8, cv.FAST_FEATURE_DETECTOR_TYPE_7_12 and cv.FAST_FEATURE_DETECTOR_TYPE_9_16

fast.setNonmaxSuppression(0)

- disables non maxima suppression

fast.detect(img, None) parameters:

- **img** - Input image. It should be grayscale



© FAST Algorithm for Corner Detection, OpenCV docs, 2022, https://docs.opencv.org/3.4/df/d0c/tutorial_py_fast.html

BRIEF (Binary Robust Independent Elementary Features)

Проблемы с SIFT и SURF при сопоставлении признаков (feature matching):

- **SIFT** использует **128-мерный вектор для дескрипторов**. Поскольку он использует числа с плавающей запятой, это занимает в основном **512 байт**.
- **SURF** требует минимум **256 байт** (для **64-мерного вектора x 4 байта** для **числа с плавающей запятой**).
- Создание таких векторов для **тысяч признаков** требует большого количества памяти, что не подходит для приложений с ограниченными ресурсами, особенно для встроенных систем.
- Чем больше занимаемая память, тем больше времени требуется для сопоставлений.

- Но все эти измерения могут не понадобиться для фактического сопоставления. Мы можем **сжать** их, используя методы, такие как **PCA**, **LDA, хеширование с использованием LSH** (Locality Sensitive Hashing), для преобразования этих дескрипторов SIFT с числами с плавающей запятой в бинарные строки. Эти бинарные строки используются для сопоставления признаков с использованием **расстояния Хэмминга**. Это обеспечивает лучшее ускорение, так как нахождение расстояния Хэмминга просто применяет операцию XOR и подсчет битов, которые очень быстры на современных процессорах с инструкциями SSE. Однако **сначала нужно найти дескрипторы**, **а затем** только мы можем **применить хеширование**, что **не решает** нашу начальную **проблему с памятью**.

- Важно, что **BRIEF** является дескриптором признаков, а не детектором признаков. Поэтому сперва нужно использовать другие детекторы признаков, такие как **SIFT**, **SURF** и т. д. В оригинальной статье рекомендуется использовать **CenSurE**, который является быстрым детектором, и BRIEF работает даже немного лучше для точек CenSurE, чем для точек SURF.

- **Идея BRIEF**: Заменить огромные массивы чисел на цепочку из нулей и единиц (бинарную строку). Нам больше не нужны сложные математические преобразования типа PCA или LDA для сжатия признаков.
- **Как это работает:** Алгоритм случайным образом выбирает пары пикселей вокруг точки и просто сравнивает их яркость: если первый пиксель ярче второго, пишем 1, если нет — 0.
- **Супер-скорость сравнения**: Вместо долгого подсчета Евклидова расстояния, компьютеру теперь достаточно применить побитовую операцию **XOR (исключающее ИЛИ) + подсчёт единиц (расстояние Хэмминга)**. Современные процессоры делают это аппаратно за один такт.

© BRIEF (Binary Robust Independent Elementary Features), OpenCV docs, 2022, https://docs.opencv.org/3.4/dc/d7d/tutorial_py_brief.html

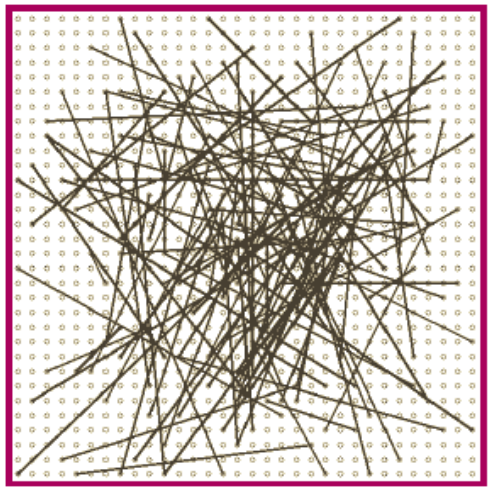
BRIEF descriptor [Calonder et. al, ECCV 2010]

- **BRIEF : B**inary **R**obust **I**ndependent **E**lementary **F**eatures
- Goal: high speed (in description and matching)

- **Binary** descriptor formation:
  - Smooth image
  - **for each** detected keypoint (e.g. FAST),

    sample all intensity pairs (I_1, I_2) (typically 256 pairs) according to pattern around the keypoint

    for each pair p
    - **if** $I_1 < I_2$ **then** ***set*** bit **p** of descriptor to **1**
    - **else *set*** bit **p** of descriptor to **0**



Pattern for intensity pair samples – generated randomly

- Not scale/rotation invariant (extensions exist…)
- Allows **very fast** Hamming Distance matching: counting the no. different bits in the descriptors tested

Дескриптор BRIEF: превращает область вокруг точки в строку из нулей и единиц.

- **Генерация битов:** Вокруг найденной ключевой точки (например, через FAST) выбираются пары пикселей I_x и I_y. Обычно берется **256 пар**.
- **Правило:** Если яркость пикселя $I_x > I_y$, в паспорт точки записывается 1, иначе — 0. Получается компактная бинарная строка длиной 256 бит.
- **Недостаток:** Оригинальный BRIEF **не инвариантен к поворотам и изменению масштаба**. Если картинку повернуть, случайные пары пикселей укажут на совершенно другие места, и дескриптор сломается.

© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

BRIEF with STAR (CenSurE) in OpenCV

Function	Description
cv2.xfeatures2d.BriefDescriptorExtractor_create()	initiates BRIEF extractor
star.detect(img,None)	detects the keypoints with STAR feature detector derived from CenSurE
brief.compute(img, kp)	computes the descriptors with BRIEF

```
import numpy as np
import cv2 as cv
from matplotlib import pyplot as plt

img = cv.imread('simple.jpg',0)

# Initiate FAST detector
star = cv.xfeatures2d.StarDetector_create()

# Initiate BRIEF extractor
brief = cv.xfeatures2d.BriefDescriptorExtractor_create()

# find the keypoints with STAR
kp = star.detect(img,None)

# compute the descriptors with BRIEF
kp, des = brief.compute(img, kp)

print( brief.descriptorSize() )
print( des.shape )
```

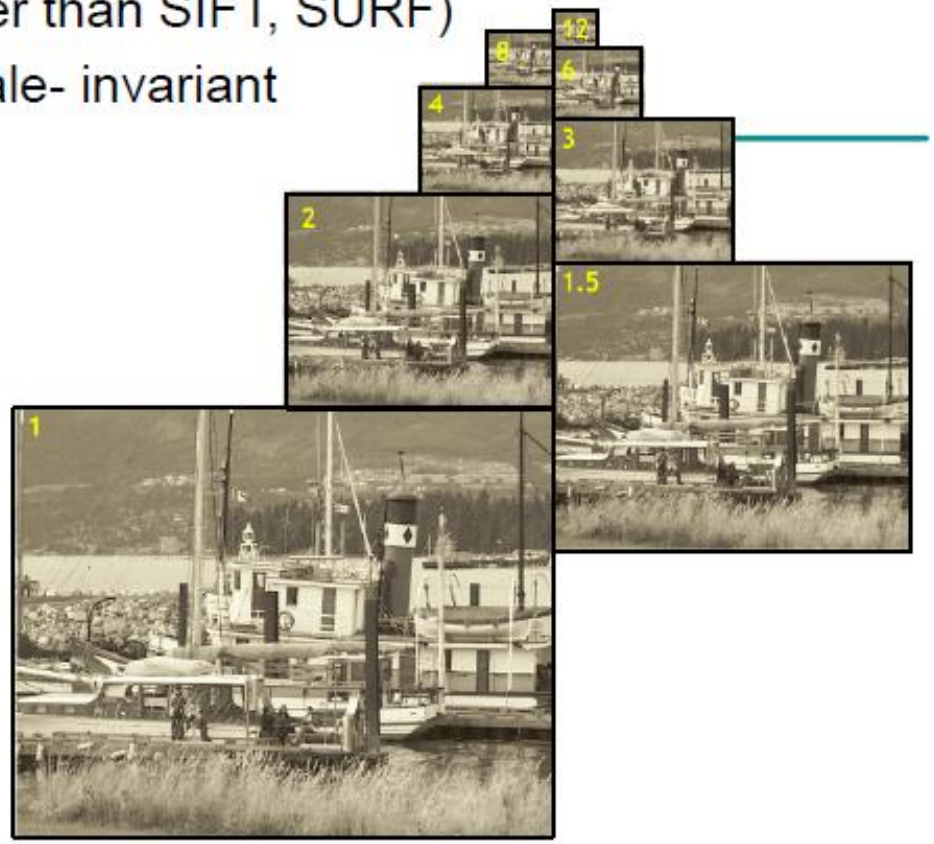
STAR (CenSurE) in OpenCV

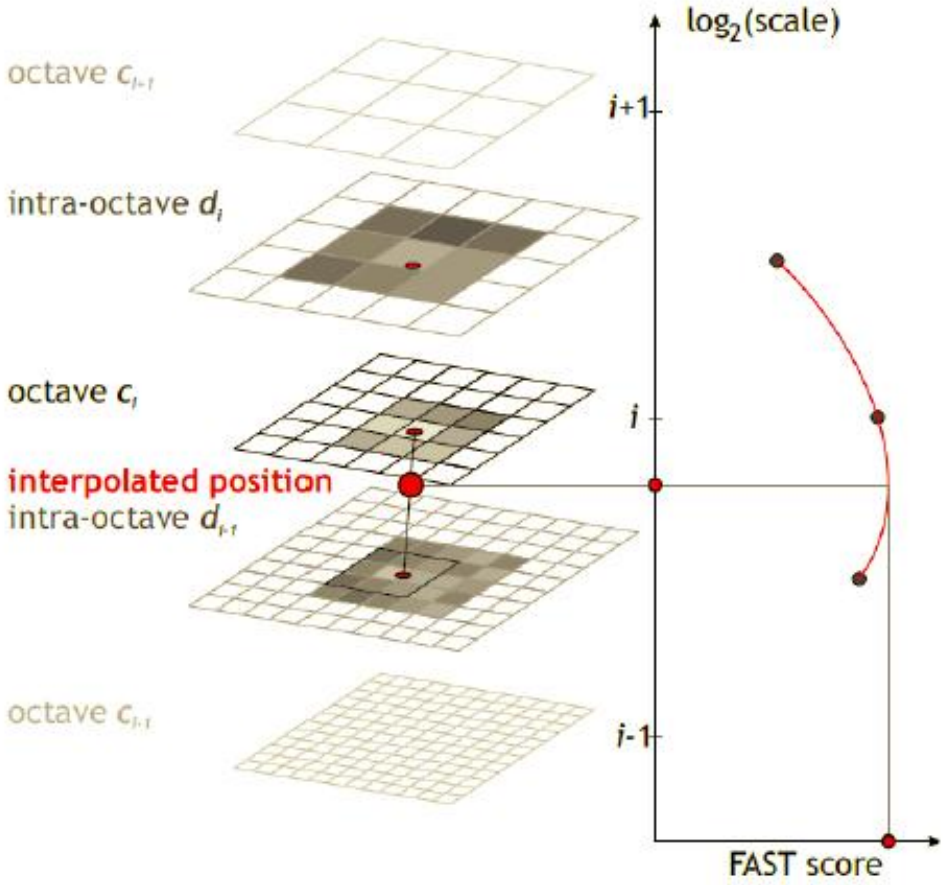
- STAR is a feature detector derived from CenSurE.
- Unlike CenSurE however, which uses polygons like squares, hexagons and octagons to approach a circle, Star emulates a circle with 2 overlapping squares: 1 upright and 1 45-degree rotated.
- These polygons are bi-level. They can be seen as polygons with thick borders. The borders and the enclosed area have weights of opposing signs.
- This has better computational characteristics than other scale-space detectors and it is capable of real-time implementation.
- In contrast to SIFT and SURF, which find extrema at sub-sampled pixels that compromises accuracy at larger scales, CenSurE creates a feature vector using full spatial resolution at all scales in the pyramid.

© BRIEF (Binary Robust Independent Elementary Features), OpenCV docs, 2022, https://docs.opencv.org/3.4/dc/d7d/tutorial_py_brief.html

BRISK features [Leutenegger, Chli, Siegwart, ICCV, 2011]

- BRISK: Binary Robust Invariant Scalable Keypoints
- Detects corners in scale-space based on FAST detection
- High-speed (faster than SIFT, SURF)
- Rotation- and scale- invariant

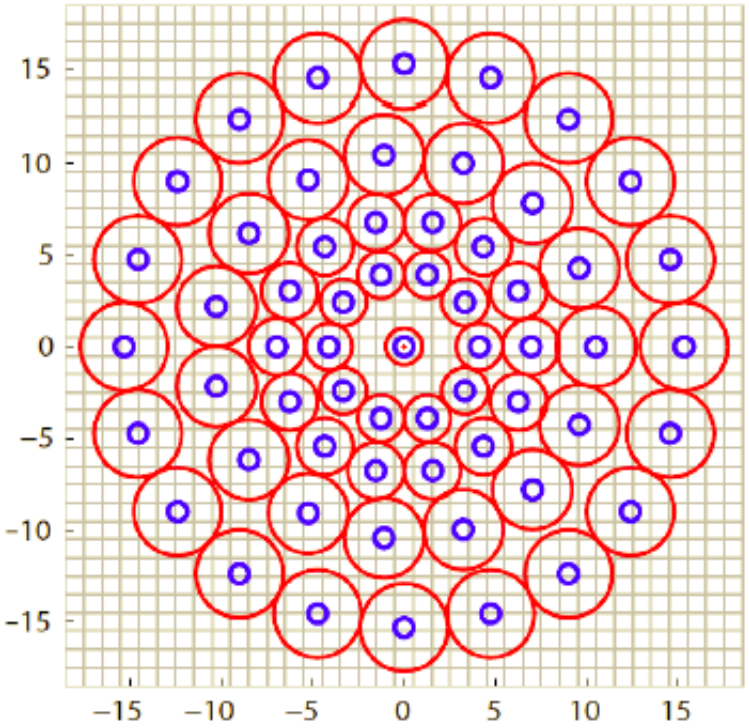




© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

BRISK features: descriptor

- **Binary**, formed by pairwise intensity comparisons (like BRIEF)
- **Pattern** defines intensity comparisons in the keypoint neighborhood
- Red circles: size of the smoothing kernel applied
- Blue circles: smoothed pixel value used
- Compare short- and long-distance pairs for orientation assignment & descriptor formation
- Detection and descriptor speed: ≈10 times faster than SURF (and even faster than SIFT)
- Slower than BRIEF, but scale- and rotation- invariant

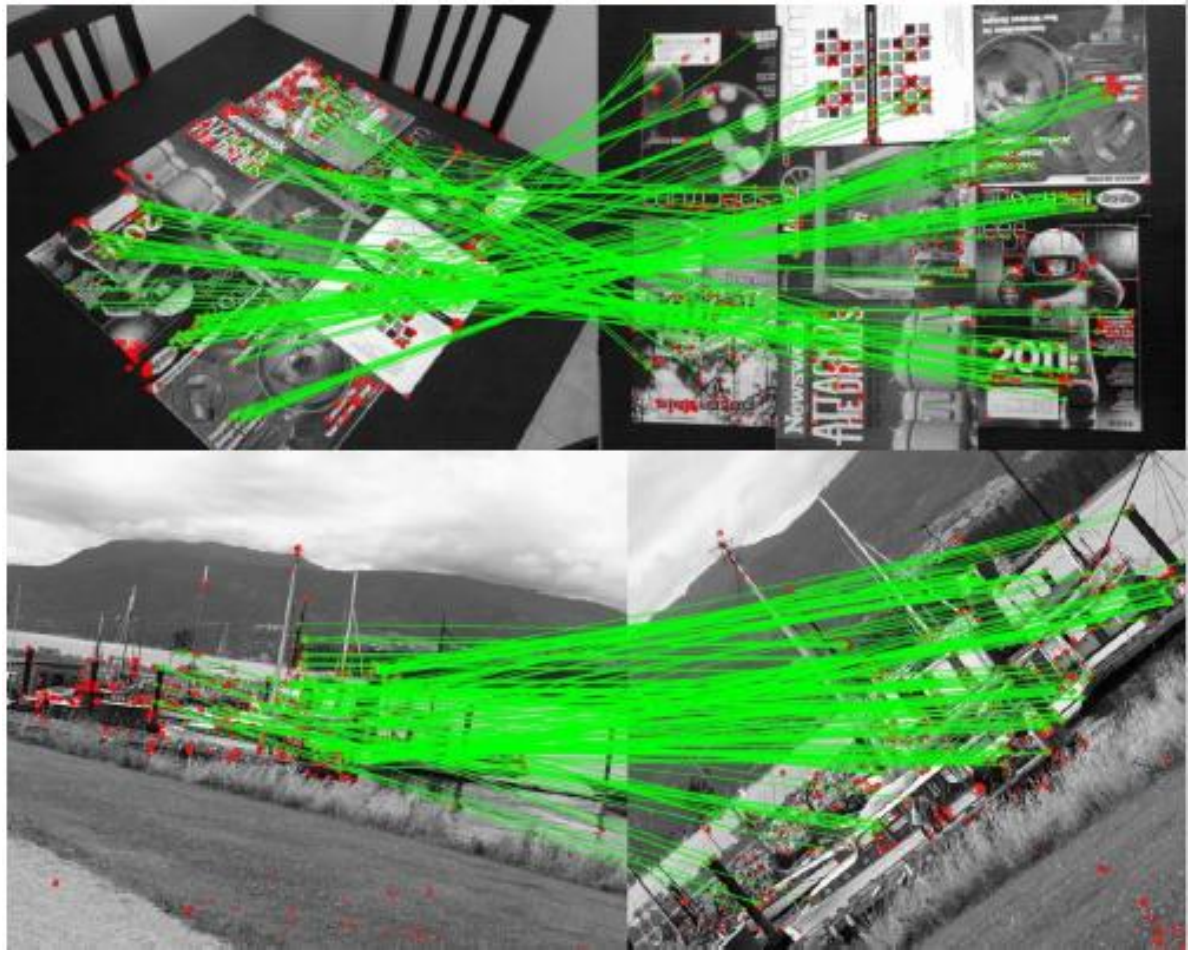


BRISK sampling pattern

© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

ORB descriptor [Rublee et al., ICCV 2011]

- ***O***riented FAST and ***R***otated ***B***RIEF
- Keypoint detector based on **FAST**
- **BRIEF** descriptors are *steered* according to keypoint orientation (to provide rotation invariance)
- Good Binary features are learned by minimizing the correlation on a set of training patches.



- **ORB:** Это гибрид модифицированного детектора FAST и дескриптора BRIEF.
- **Управление дескриптором:** Алгоритм вычисляет угол поворота ключевой точки и «поворачивает» бинарные тесты BRIEF вслед за этим углом. Это даёт инвариантность к вращению..
- **Обучение признаков:** Авторы применили машинное обучение на тренировочном наборе картинок, чтобы отобрать только те пары пикселей, которые дают самую высокую контрастность и наименьшую зависимость друг от друга.

© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

ORB (Oriented FAST and Rotated BRIEF)

- Этот алгоритм был разработан Итаном Рубли, Винсентом Рабо, Куртом Конолигом и Гэри Р. Брадски из "OpenCV Labs" в их статье "ORB: An efficient alternative to SIFT or SURF" в 2011 году. Он является **хорошей альтернативой SIFT и SURF** с точки зрения **вычислительных затрат**, **производительности сопоставления** и, в основном, **патентов** (так как SIFT и SURF запатентованы).
- **ORB (Oriented FAST and Rotated BRIEF)** в основном представляет собой сочетание детектора ключевых точек FAST и дескриптора BRIEF с множеством модификаций для повышения производительности.
    1. Сначала используется **FAST для нахождения ключевых точек**, затем применяется **мера углов Харриса для нахождения лучших N точек среди них**. Также используется **пирамида для создания многоуровневых признаков**.
    2. **Поскольку FAST не вычисляет ориентацию**, авторы предложили следующую модификацию для достижения инвариантности к поворотам: **вычисляется интенсивность взвешенного центроида области с расположенным в центре углом**. **Направление вектора от этой угловой точки к центроиду дает ориентацию**. Для улучшения инвариантности к поворотам моменты вычисляются по x и y, которые должны находиться в круговой области радиусом r.

- **Детектор oFAST (Oriented FAST)**: Оригинальный FAST не умеет определять масштаб и поворот. ORB решает это так:
    1. Строит пирамиду изображений для поиска масштаба.
    2. Находит угол поворота с помощью **интенсивностного центроида** (вектора, направленного из геометрического центра круга в сторону его «центра масс» по яркости).

- **Дескриптор rBRIEF (Rotated BRIEF)**: Обычные случайные тесты BRIEF при повороте теряют свою различительную способность (дисперсию). В rBRIEF используется специальный жадный алгоритм, который из всех возможных пар пикселей выбрал **256 идеальных тестов**, имеющих среднее значение отклика, близкое к 0.5, и высокую дисперсию.
- **Мгновенное сопоставление**: Для сравнения точек используются **multi-probe LSH** (локально-чувствительное хеширование), что позволяет находить соответствия на огромной скорости даже на недорогих смартфонах и роботах.

© ORB (Oriented FAST and Rotated BRIEF), OpenCV docs, 2022, https://docs.opencv.org/3.4/d1/d89/tutorial_py_orb.html

ORB (Oriented FAST and Rotated BRIEF) in OpenCV

Function	Description
cv2.ORB_create()	initiates ORB detector
orb.detect(img,None)	detects keypoints with ORB
orb.compute(img, kp)	computes the descriptors with ORB

```
import numpy as np
import cv2 as cv
from matplotlib import pyplot as plt

img = cv.imread('simple.jpg',0)

# Initiate ORB detector
orb = cv.ORB_create()

# find the keypoints with ORB
kp = orb.detect(img,None)

# compute the descriptors with ORB
kp, des = orb.compute(img, kp)

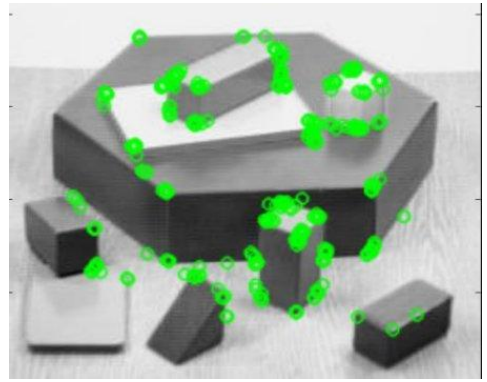
# draw only keypoints location,not size and orientation
img2 = cv.drawKeypoints(img, kp, None, color=(0,255,0), flags=0)
plt.imshow(img2), plt.show()
```

Initially, we have to create an ORB object with the function, cv.ORB() or using feature2d common interface. It has a number of optional parameters. Most useful ones are

- **nFeatures** – denotes maximum number of features to be retained (by default 500)
- **scoreType** – denotes whether Harris score or FAST score to rank the features (by default, Harris score) etc.
- **WTA_K** - decides number of points that produce each element of the oriented BRIEF descriptor,

WTA_K=2 (by default), i.e. It selects two points at a time. In that case, for matching, NORM_HAMMING distance is used.

WTA_K=3 or **4**, it takes 3 or 4 points to produce BRIEF descriptor, then matching distance is defined by NORM_HAMMING2.



© ORB (Oriented FAST and Rotated BRIEF), OpenCV docs, 2022, https://docs.opencv.org/3.4/d1/d89/tutorial_py_orb.html

Recap table

The distinctiveness is also the main difference between Harris and SIFT features. Harris privileges corners (such as edge intersections), while SIFT privileges image patches with highly informative content (i.e., not corners).

Detector	Descriptor	Localization Accuracy	Relocalization & Loop closing	Efficiency
Harris	Patch	++++	+	+++
Shi-Tomasi	Patch	++++	+	+++
SIFT	SIFT	+++	++++	+
SURF	SURF	+++	++++	++
FAST	BRIEF ORB BRISK	++++	+++	++++

© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

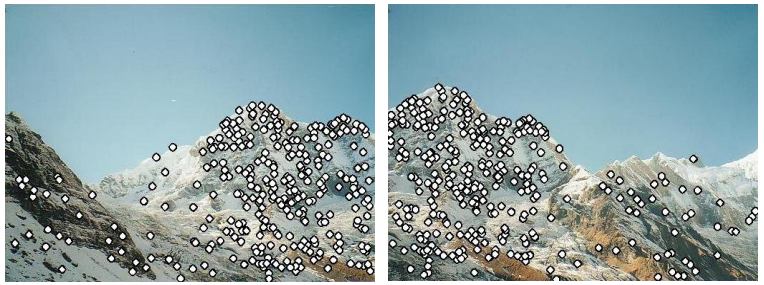
3 Сопоставление признаков в OpenCV

Feature matching · Евклидово расстояние · косинусное сходство · Ближайший сосед (nearest neighbor) · нейросетевой дескриптор (LIFT)

Local features: main components

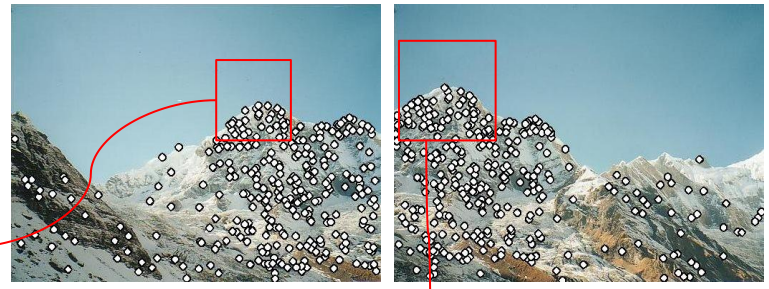
1) Detection:

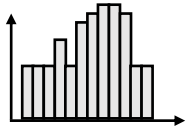
Find a set of distinctive key points.

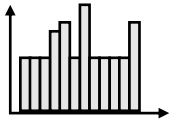


2) Description:

Extract feature descriptor around each interest point as vector.



x_1  $\mathbf{x}_1 = [x_1^{(1)}, \ldots, x_d^{(1)}]$

x_2  $\mathbf{x}_2 = [x_1^{(2)}, \ldots, x_d^{(2)}]$

3) Matching:

Compute distance between feature vectors to find correspondence.

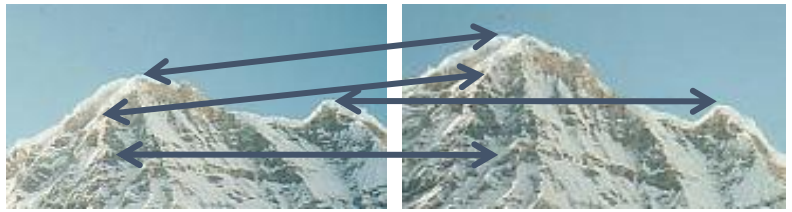
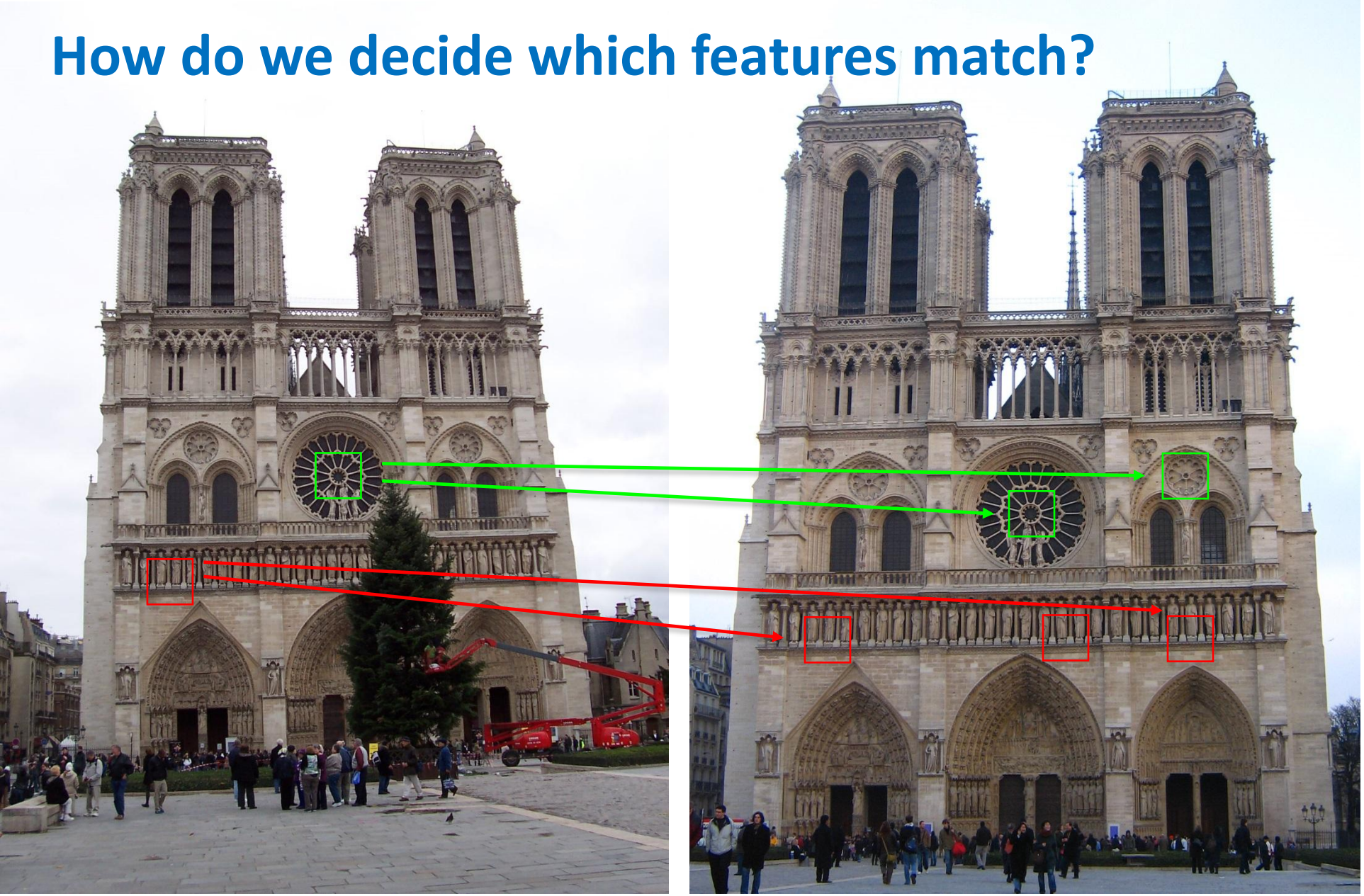


Fig. © K. Grauman, B. Leibe

How do we decide which features match?



Distance: 0.34, 0.30, 0.40

Distance: 0.61, 1.22

Fig. © K. Grauman, B. Leibe

Euclidean distance vs. Cosine Similarity

Euclidean distance:

$$\mathrm{d}(\mathbf{p}, \mathbf{q}) = \mathrm{d}(\mathbf{q}, \mathbf{p}) = \sqrt{(q_1 - p_1)^2 + (q_2 - p_2)^2 + \cdots + (q_n - p_n)^2}$$

$$= \sqrt{\sum_{i=1}^{n} (q_i - p_i)^2}.$$

Cosine similarity:

$$\mathbf{a} \cdot \mathbf{b} = \|\mathbf{a}\|_2 \|\mathbf{b}\|_2 \cos\theta$$

$$\text{similarity} = \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\|_2 \|\mathbf{B}\|_2}$$

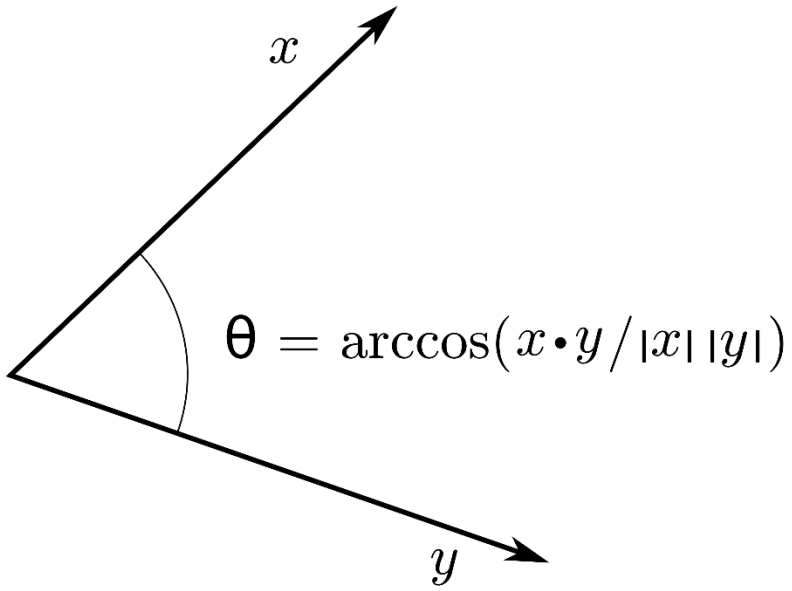


Fig. © Wikipedia

Feature Matching

Критерий 1:

1. Вычислить расстояние в пространстве признаков, например, евклидово расстояние между 128-мерными дескрипторами SIFT.
   - ❑ Измеряется расстояние между 128-мерными векторами
2. Найти точки соответствия с наименьшим расстоянием (ближайший сосед)
   - ❑ После вычисления расстояния для всех пар дескрипторов, точка с наименьшим расстоянием считается наиболее похожей (наиболее близкой) и, следовательно, соответствующей

Проблема:

У всего ли есть соответствие?

- ❑ Один из вопросов, который возникает при таком подходе, заключается в том, что не каждое найденное соответствие может быть правильным. Не все точки на одном изображении обязательно имеют точные соответствия на другом изображении
- ❑ Возможно, что некоторые дескрипторы не будут иметь подходящих пар из-за шума, изменений в изображении или других факторов

Fig. © Artur Sagitov, Algorithms and Systems of Computer Vision, SPBPU, 2022

Feature Matching

Критерий 2:

1. Вычислить расстояние в пространстве признаков, например, евклидово расстояние между 128-мерными дескрипторами SIFT
   - ❑ Измеряется расстояние между 128-мерными векторами
2. Найти точки соответствия с наименьшим расстоянием (ближайший сосед)
   - ❑ После вычисления расстояния для всех пар дескрипторов, точка с наименьшим расстоянием считается наиболее похожей (наиболее близкой) и, следовательно, соответствующей
3. Игнорировать все, что выше порога (нет соответствия!)
   - ❑ Если расстояние между признаками превышает установленный порог, такие точки не считаются соответствующими

Проблемы:

1. Сложность выбора порога
   - ❑ Если порог слишком низкий, многие правильные соответствия будут пропущены. Если порог слишком высокий, будет много ложных срабатываний.
2. Недистинктивные признаки могут иметь множество близких соответствий, из которых только одно будет правильным
   - ❑ Признаки, которые не являются уникальными (недистинктивные), могут иметь много близких соответствий, что затрудняет правильное определение нужного соответствия. Только одно из этих соответствий может быть правильным, остальные будут ложными срабатываниями.

Fig. © Artur Sagitov, Algorithms and Systems of Computer Vision, SPBPU, 2022

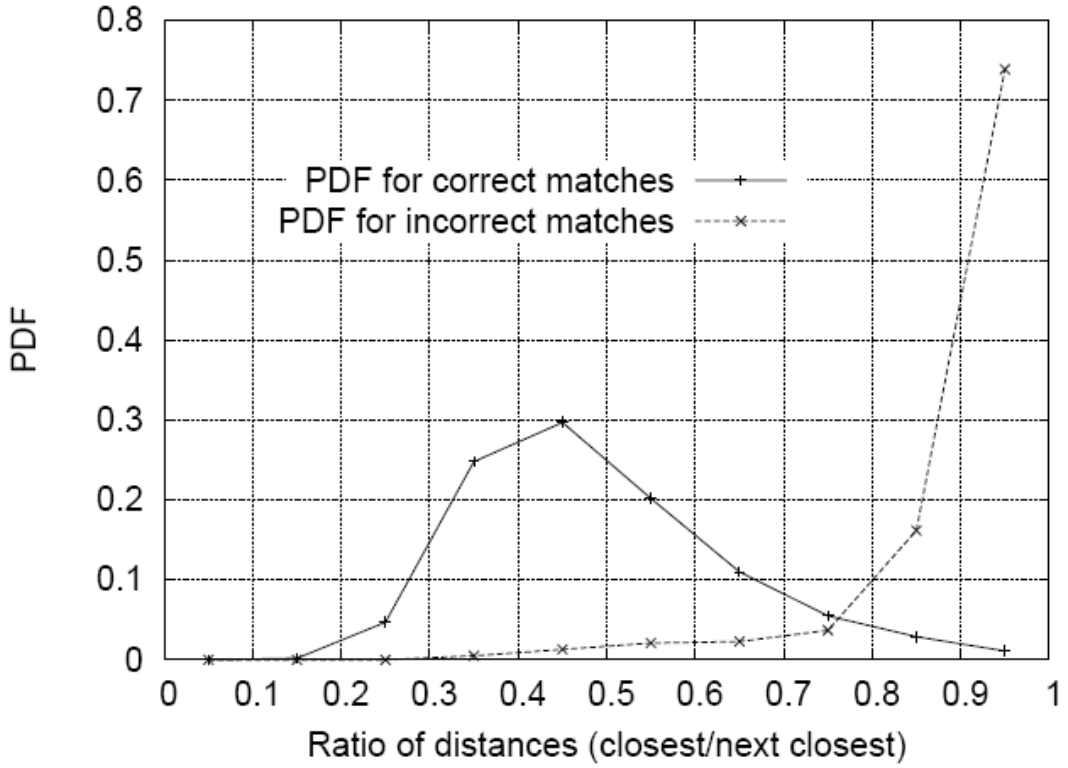
Nearest Neighbor Distance Ratio

1. Сравнение расстояния до ближайшего и второго ближайшего соседа:
   - ❑ Чтобы определить, насколько уникальны и надежны соответствия, сравниваются расстояния до ближайшего (NN1) и второго ближайшего (NN2) соседа вектора признаков.
2. Вычисление коэффициента *NN1/NN2*:
   - ❑ Если ближайший и второй ближайший соседи имеют почти одинаковые расстояния до точки (NN1 ≈ NN2), то коэффициент $NN1/NN2$ будет примерно равен 1. Это указывает на то, что соответствия слишком близки, и их нельзя считать надежными.
   - ❑ Если расстояние до ближайшего соседа значительно меньше, чем до второго ближайшего (NN1 << NN2), то коэффициент $NN1/NN2$ стремится к 0. Это означает, что ближайший сосед является более надежным и уникальным соответствием.
3. Сортировка по коэффициенту *NN1/NN2*:
   - ❑ Сортировка соответствий по этому коэффициенту позволяет расставить их по степени уверенности. Более низкие значения коэффициента указывают на более надежные и уверенные соответствия.
4. Выбор порогового коэффициента:
   - ❑ Один из вопросов заключается в выборе порогового значения коэффициента. Это значение определяет, какие соответствия будут считаться надежными, а какие нет.
   - ❑ Выбор порога зависит от конкретной задачи и данных, и может потребовать экспериментов для определения оптимального значения.

Fig. © Artur Sagitov, Algorithms and Systems of Computer Vision, SPBPU, 2022

Nearest Neighbor Distance Ratio

- Лоу (David Lowe), разработчик алгоритма SIFT, анализировал вероятности отношений расстояний между ближайшими и вторыми ближайшими соседями (NN1 и NN2) для дескрипторов признаков. Это позволило определить, насколько надежно каждое соответствие.
- Для оценки и обучения алгоритма Лоу использовал большой набор данных, содержащий 40,000 ключевых точек, которые были вручную размечены. Истинные данные (ground truth) — это эталонные значения, которые используются для сравнения и оценки точности алгоритмов.

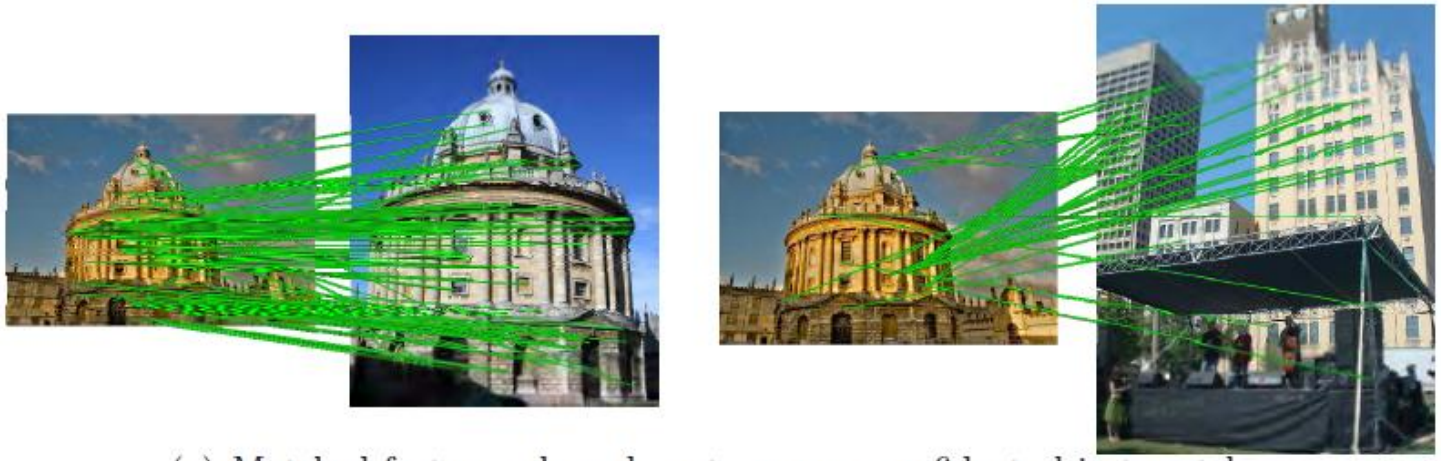


Порог отношения *NN1/NN2* зависит от вашего взгляда на баланс между количеством ложноположительных и истинно положительных срабатываний в вашем приложении.

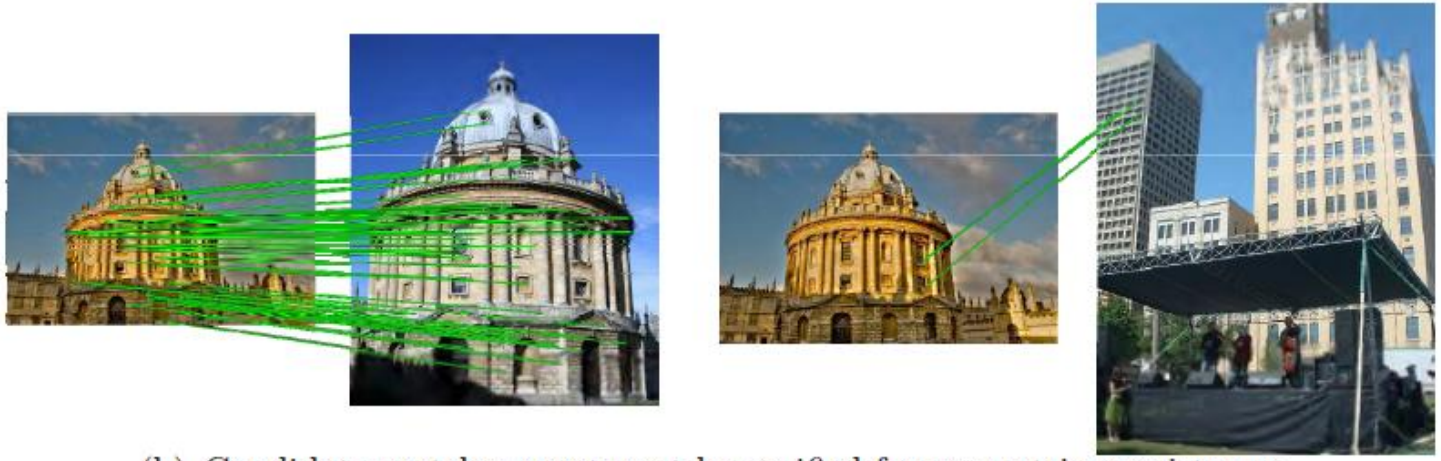
Вероятность того, что соответствие является правильным, можно определить, взяв отношение расстояния до ближайшего соседа к расстоянию до второго ближайшего. Используя базу данных из 40,000 ключевых точек, сплошная линия показывает функцию плотности вероятности (PDF) этого отношения для правильных соответствий, а пунктирная линия для неправильных соответствий.

Fig. © Lowe IJCV 2004

Feature matching does not guarantee object matching



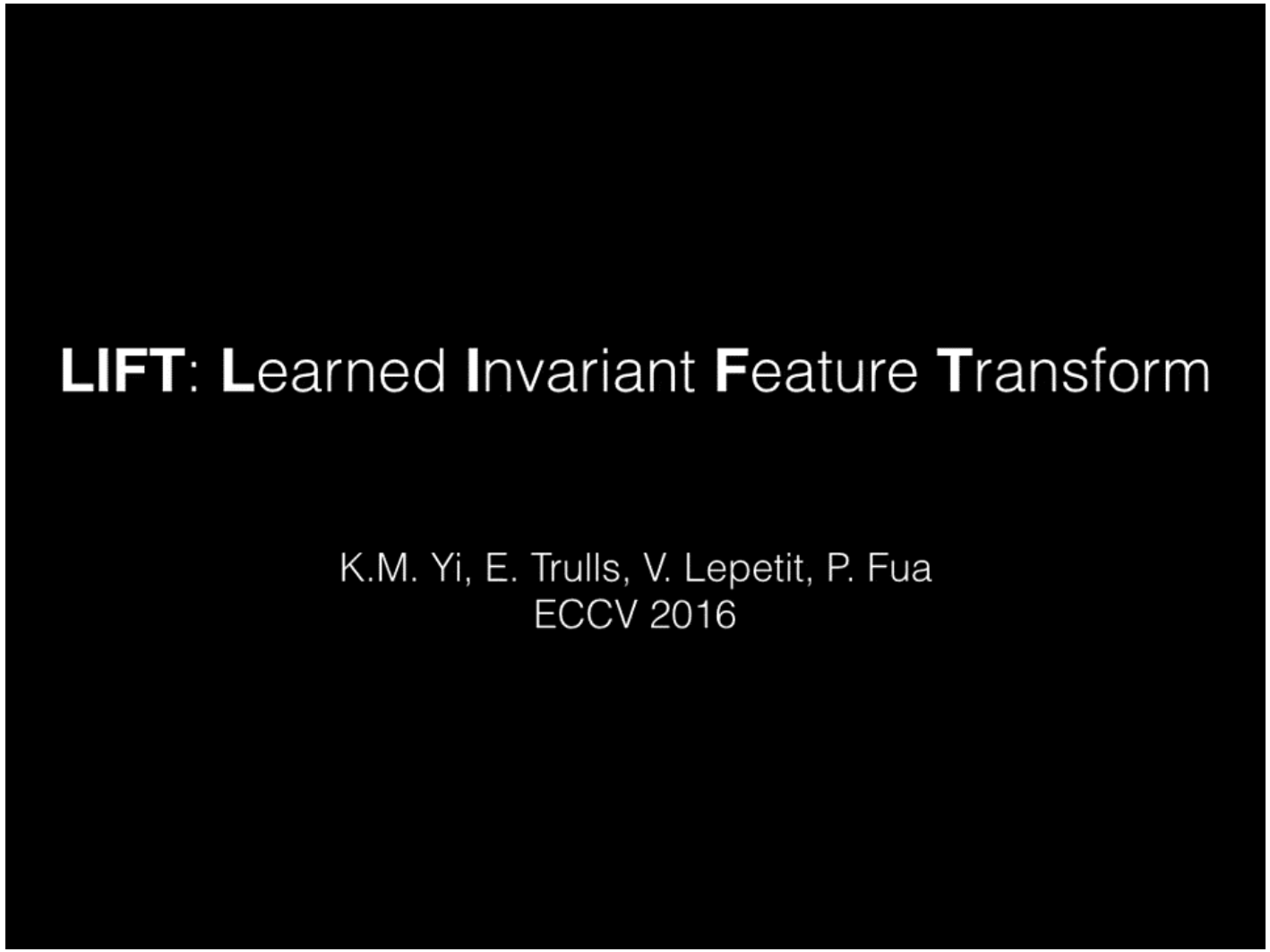
(a) Matched features alone do not ensure a confident object match.



(b) Candidate matches must next be verified for geometric consistency.

Fig. © Artur Sagitov, Algorithms and Systems of Computer Vision, SPBPU, 2022

Learned Invariant Feature Transform (LIFT)



© Sculdo, K.M. Yi, E. Trullis, V. Lepetit, P. Fua, LIFT: Learned Invariant Feature Transform, ECCV, 2016, https://www.youtube.com/watch?v=hhxAttChmCo

Brute-Force Matcher with ORB descriptors

- **Brute-Force matcher** прост. Он берет дескриптор одного признака из первого набора и сопоставляет его со всеми другими признаками из второго набора с использованием некоторого вычисления расстояния. Возвращается самый близкий.

Function	Description
cv2.BFMatcher()	creates Brute Force matcher object
cv2.BFMatcher.match(des1,des2)	matches descriptors
cv2.drawMatches	draws the matches

```
import numpy as np
import cv2 as cv
import matplotlib.pyplot as plt

img1 = cv.imread('box.png',cv.IMREAD_GRAYSCALE)          # queryImage
img2 = cv.imread('box_in_scene.png',cv.IMREAD_GRAYSCALE) # trainImage

# Initiate ORB detector
orb = cv.ORB_create()

# find the keypoints and descriptors with ORB
kp1, des1 = orb.detectAndCompute(img1,None)
kp2, des2 = orb.detectAndCompute(img2,None)
```

```
# create BFMatcher object
bf = cv.BFMatcher(cv.NORM_HAMMING, crossCheck=True)

# Match descriptors.
matches = bf.match(des1,des2)

# Sort them in the order of their distance.
matches = sorted(matches, key = lambda x:x.distance)

# Draw first 10 matches.
img3 = cv.drawMatches(img1,kp1,img2,kp2,matches[:10],None,flags=cv.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS)

plt.imshow(img3),plt.show()
```

cv2.BFMatcher(normType, crossCheck)

- **normType** – specifies the distance measurement to be used. By default, it is cv.NORM_L2. It is good for SIFT, SURF etc. For binary string based descriptors like ORB, BRIEF, BRISK etc, cv.NORM_HAMMING should be used
- **crossCheck** - boolean variable, if it is true, Matcher returns only those matches with value (i,j) such that i-th descriptor in set A has j-th descriptor in set B as the best match and vice-versa.



© Feature Matching, OpenCV docs, 2022, https://docs.opencv.org/3.4/dc/dc3/tutorial_py_matcher.html

Brute-Force Matching with SIFT Descriptors

- **Brute-Force matcher** прост. Он берет дескриптор одного признака из первого набора и сопоставляет его со всеми другими признаками из второго набора с использованием некоторого вычисления расстояния. Возвращается самый близкий.

Function	Description
cv2.BFMatcher.knnMatch()	returns k best matches (specified by the user)
cv2.drawMatchesKnn()	draws all the k best matches. If k=2, it will draw two match-lines for each keypoint.

```python
import numpy as np
import cv2 as cv
import matplotlib.pyplot as plt

img1 = cv.imread('box.png',cv.IMREAD_GRAYSCALE)          # queryImage
img2 = cv.imread('box_in_scene.png',cv.IMREAD_GRAYSCALE) # trainImage

# Initiate SIFT detector
sift = cv.SIFT_create()

# find the keypoints and descriptors with SIFT
kp1, des1 = sift.detectAndCompute(img1,None)
kp2, des2 = sift.detectAndCompute(img2,None)

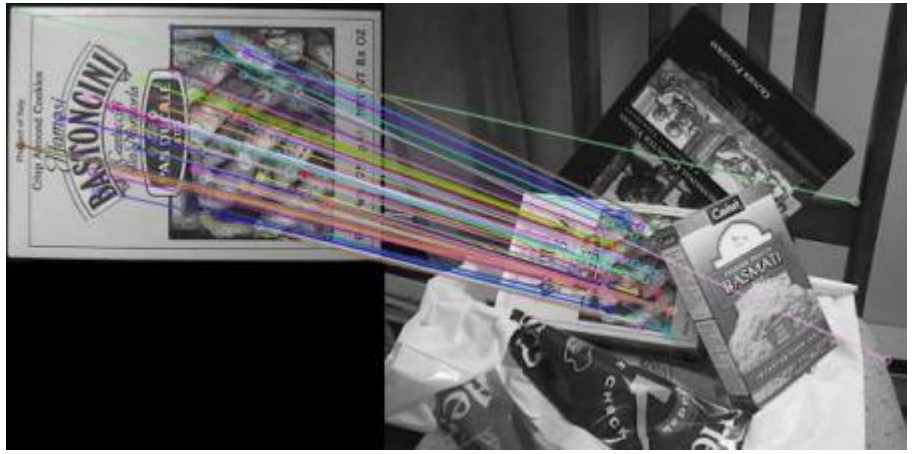
# BFMatcher with default params
bf = cv.BFMatcher()
matches = bf.knnMatch(des1,des2,k=2)

# Apply ratio test
good = []
for m,n in matches:
    if m.distance < 0.75*n.distance:
        good.append([m])

# cv.drawMatchesKnn expects list of lists as matches.
img3 = cv.drawMatchesKnn(img1,kp1,img2,kp2,good,None,flags=cv.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS)

plt.imshow(img3),plt.show()
```

we are using BFMatcher.knnMatch() to get k best matches. In this example, we will take k=2 so that we can apply ratio test explained by D. Lowe in SIFT paper



© Feature Matching, OpenCV docs, 2022, https://docs.opencv.org/3.4/dc/dc3/tutorial_py_matcher.html

FLANN based Matcher

- **FLANN** означает Быстрая библиотека для приближенного поиска ближайших соседей (Fast Library for Approximate Nearest Neighbors). Она содержит набор алгоритмов, оптимизированных для быстрого поиска ближайших соседей в больших наборах данных и для высокоразмерных признаков. Она работает быстрее, чем BFMatcher для больших наборов данных.

```python
import numpy as np
import cv2 as cv
import matplotlib.pyplot as plt

img1 = cv.imread('box.png',cv.IMREAD_GRAYSCALE)          # queryImage
img2 = cv.imread('box_in_scene.png',cv.IMREAD_GRAYSCALE) # trainImage

# Initiate SIFT detector
sift = cv.SIFT_create()

# find the keypoints and descriptors with SIFT
kp1, des1 = sift.detectAndCompute(img1,None)
kp2, des2 = sift.detectAndCompute(img2,None)

# FLANN parameters
FLANN_INDEX_KDTREE = 1
index_params = dict(algorithm = FLANN_INDEX_KDTREE, trees = 5)
search_params = dict(checks=50)   # or pass empty dictionary

flann = cv.FlannBasedMatcher(index_params,search_params)

matches = flann.knnMatch(des1,des2,k=2)

# Need to draw only good matches, so create a mask
matchesMask = [[0,0] for i in range(len(matches))]

# ratio test as per Lowe's paper
for i,(m,n) in enumerate(matches):
    if m.distance < 0.7*n.distance:
        matchesMask[i]=[1,0]

draw_params = dict(matchColor = (0,255,0),
                   singlePointColor = (255,0,0),
                   matchesMask = matchesMask,
                   flags = cv.DrawMatchesFlags_DEFAULT)

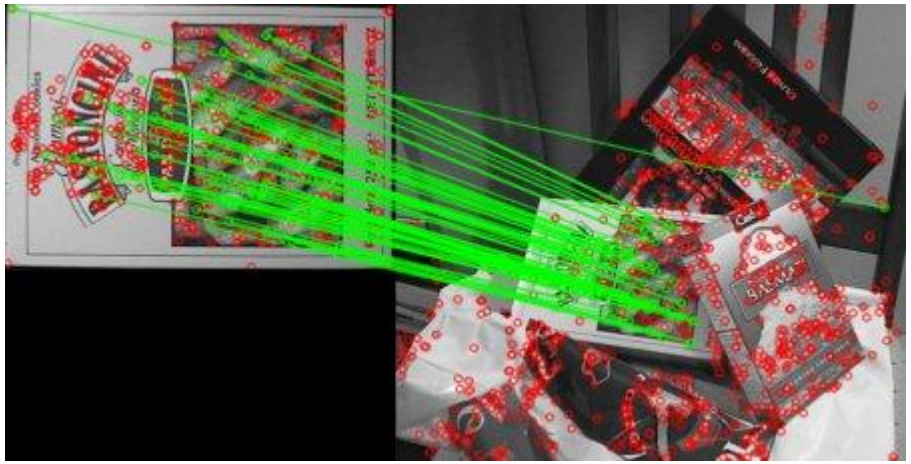
img3 = cv.drawMatchesKnn(img1,kp1,img2,kp2,matches,None,**draw_params)

plt.imshow(img3,),plt.show()
```

Function	Description
cv2.FlannBasedMatcher()	creates FLANN matcher object with two dictionaries which specifies the algorithm to be used, its related parameters etc.
cv2.FlannBasedMatcher.knnMatch()	returns k best matches (specified by the user)

cv2.FlannBasedMatcher(normType, crossCheck)

- **IndexParams** – For SIFT and SURF, dict(algorithm = FLANN_INDEX_KDTREE, trees = 5)
- **SearchParams** - specifies the number of times the trees in the index should be recursively traversed. Higher values gives better precision, but also takes more time



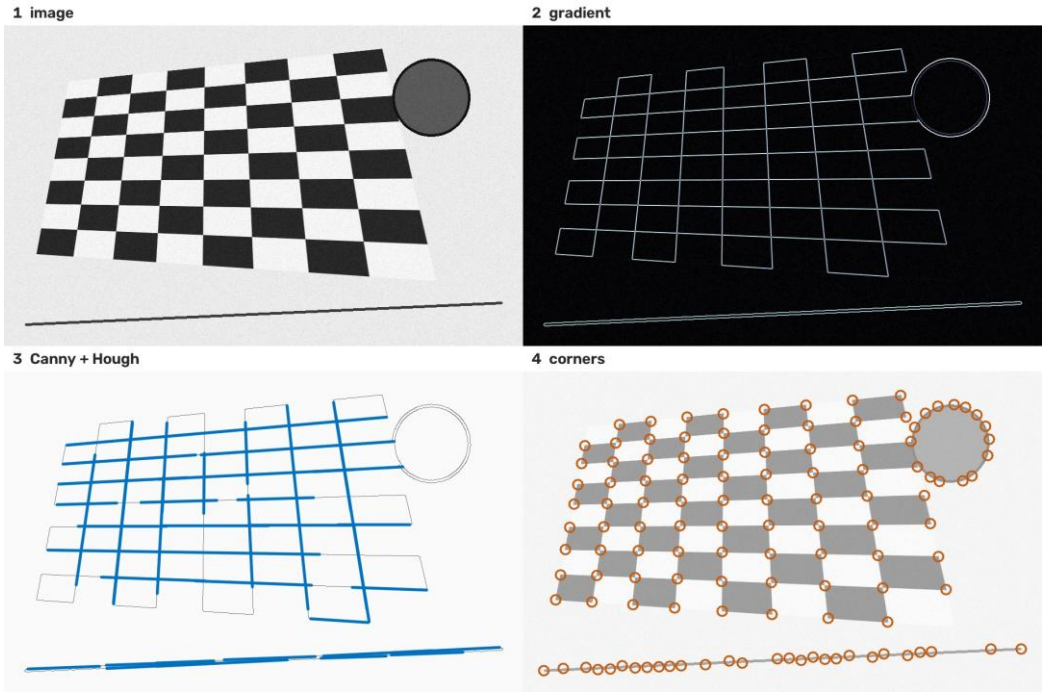
© Feature Matching, OpenCV docs, 2022, https://docs.opencv.org/3.4/dc/dc3/tutorial_py_matcher.html

References

REFERENCES

- Szeliski R. Computer Vision: Algorithms and Applications, 2nd ed., 2022 — §7.2 (края), §7.4 (прямые и робастное оценивание); к L8 — §7.1 (точки интереса)
- Canny J. A Computational Approach to Edge Detection, IEEE TPAMI, 1986 · Duda R., Hart P. Use of the Hough Transform to Detect Lines and Patterns, CACM, 1972
- Fischler M., Bolles R. Random Sample Consensus, CACM, 1981 · Raguram R. et al. USAC: A Universal Framework for Random Sample Consensus, IEEE TPAMI, 2013
- Гонсалес Р., Вудс Р. Цифровая обработка изображений, 3-е изд., 2012 — гл. 9 (морфология), гл. 10 (сегментация по краям)
- OpenCV docs 4.x: `Canny`, `Sobel`/`Scharr`, `findContours`/`approxPolyDP`, `HoughLines`/`HoughLinesP`, `fitLine` (DIST_*), USAC-флаги RANSAC

Вопросы?



Dr. Ilya Afanasyev · i.afanasyev@innopolis.ru